# Chemical Mixtures and Accelerated Biological Aging in CKD: NHANES Analysis

Reproducible NHANES analysis By January

2026-04-25

## Contents

**Study Objective** 1

**Configuration** 2

**Packages** 3

**Helper Functions** 3

**Data Acquisition and Cycle Availability** 7

**Variable Construction: PhenoAge, CKD, Covariates** 9

**Exposure Processing and LOD Handling** 19

**Analytic Sample and Survey Design** 23

**Mixture Modeling** 26

**Sample Flow (PRISMA-style)** 37

**Tables** 39

**Figures** 54

**Cautions and Limitations** 57

**Reproducibility** 58

## Study Objective

Primary question: are environmental chemical mixtures associated with accelerated biological aging (PhenoAge acceleration), and is this association stronger among individuals with CKD?

# Configuration

```r
config <- list(
  cycles = c("2003-2004", "2005-2006", "2007-2008", "2009-2010", "2011-2012", "2013-2014", "2015-2016",
  exposure_family = "pfas", # pfas, metals, both
  q = 4,
  min_age = 20,
  ckd_def = "egfr_or_acr", # egfr_or_acr, egfr_only, acr_only
  use_residualized_phenoage = TRUE,   # PhenoAgeAccel is always residualized; option reserved for futur
  # Primary analysis uses fixed 4-PFAS set with complete-case; strict (qgcomp-compatible subset) is sen
  require_complete_exposure_set = FALSE,  # Legacy: primary is always fixed-set complete-case; no longe
  min_nonmissing_exposure_fraction = 0.6,
  min_nonmissing_exposures_floor = 2,
  include_clinical_covariates = TRUE,    # Not wired to primary (6-variable set); reserved for future s
  # Weight is resolved from audit; PFAS workflow prioritizes environmental subsample WTS* weights.
  weight_type = "auto",  # Auto-resolve from audit in the exposure-complete sample (PFAS excludes WTINT
  verbose = TRUE
)
config
```

```
## $cycles
## [1] "2003-2004" "2005-2006" "2007-2008" "2009-2010" "2011-2012" "2013-2014"
## [7] "2015-2016" "2017-2018"
##
## $exposure_family
## [1] "pfas"
##
## $q
## [1] 4
##
## $min_age
## [1] 20
##
## $ckd_def
## [1] "egfr_or_acr"
##
## $use_residualized_phenoage
## [1] TRUE
##
## $require_complete_exposure_set
## [1] FALSE
##
## $min_nonmissing_exposure_fraction
## [1] 0.6
##
## $min_nonmissing_exposures_floor
## [1] 2
##
## $include_clinical_covariates
## [1] TRUE
##
## $weight_type
```

```
## [1] "auto"
##
## $verbose
## [1] TRUE
```

## Packages

```r
required_pkgs <- c(
  "nhanesA", "dplyr", "tidyr", "purrr", "stringr", "survey", "splines",
  "ggplot2", "broom", "knitr", "qgcomp", "forcats", "tibble"
)
invisible(lapply(required_pkgs, function(p) {
  if (!requireNamespace(p, quietly = TRUE)) stop("Package not installed: ", p)
}))

library(nhanesA)
library(dplyr)
library(tidyr)
library(purrr)
library(stringr)
library(survey)
options(survey.lonely.psu = "adjust")
library(splines)
library(ggplot2)
library(broom)
library(knitr)
library(qgcomp)
library(forcats)
library(tibble)
```

## Helper Functions

```r
cycle_to_suffix <- function(cycle) {
  map <- c(
    "1999-2000" = "A", "2001-2002" = "B", "2003-2004" = "C", "2005-2006" = "D",
    "2007-2008" = "E", "2009-2010" = "F", "2011-2012" = "G", "2013-2014" = "H",
    "2015-2016" = "I", "2017-2018" = "J", "2019-2020" = "K"
  )
  out <- unname(map[cycle])
  if (is.na(out)) stop("Cycle not mapped: ", cycle)
  out
}

safe_nhanes <- function(table_name, verbose = TRUE) {
  out <- tryCatch(nhanesA::nhanes(table_name), error = function(e) NULL)
  if (is.null(out) && verbose) message("Unavailable table: ", table_name)
  out
}
```

```r
resolve_table <- function(cycle, candidates, domain = NULL, verbose = TRUE) {
  suf <- cycle_to_suffix(cycle)
  to_try <- paste0(candidates, "_", suf)
  for (tb in to_try) {
    dat <- safe_nhanes(tb, verbose = FALSE)
    if (!is.null(dat)) {
      if (verbose) message("Using table ", tb, " for cycle ", cycle)
      return(list(code = candidates[1], table = tb, data = dat))
    }
  }
  NULL
}

coalesce_num <- function(df, vars) {
  vars <- vars[vars %in% names(df)]
  if (length(vars) == 0) return(rep(NA_real_, nrow(df)))
  v <- lapply(df[vars], function(x) suppressWarnings(as.numeric(as.character(x))))
  dplyr::coalesce(!!!v)
}

parse_yes_no <- function(x) {
  x_num <- suppressWarnings(as.numeric(as.character(x)))
  x_chr <- toupper(trimws(as.character(x)))
  case_when(
    x_num == 1 ~ 1,
    x_num == 2 ~ 0,
    str_detect(x_chr, "^YES$") ~ 1,
    str_detect(x_chr, "^NO$") ~ 0,
    TRUE ~ NA_real_
  )
}

parse_smq040 <- function(x) {
  x_chr <- toupper(trimws(as.character(x)))
  x_num <- suppressWarnings(as.numeric(as.character(x)))
  case_when(
    x_num %in% c(1, 2) ~ 1,
    x_num == 3 ~ 0,
    str_detect(x_chr, "EVERY DAY|SOME DAYS") ~ 1,
    str_detect(x_chr, "^NOT AT ALL$") ~ 0,
    TRUE ~ NA_real_
  )
}

parse_phq_item <- function(x) {
  x_chr <- toupper(trimws(as.character(x)))
  x_num <- suppressWarnings(as.numeric(as.character(x)))
  case_when(
    x_num == 1 ~ 0, x_num == 2 ~ 1, x_num == 3 ~ 2, x_num == 4 ~ 3,
    x_num %in% 0:3 ~ x_num,
    str_detect(x_chr, "^NOT AT ALL$") ~ 0,
    str_detect(x_chr, "^SEVERAL DAYS$") ~ 1,
    str_detect(x_chr, "^MORE THAN HALF THE DAYS$") ~ 2,
```

```r
    str_detect(x_chr, "^NEARLY EVERY DAY$") ~ 3,
    TRUE ~ NA_real_
  )
}

yesno01 <- function(x) {
  xc <- toupper(trimws(as.character(x)))
  xn <- suppressWarnings(as.numeric(as.character(x)))
  case_when(
    xn == 1 ~ 1,
    xn == 2 ~ 0,
    xc %in% c("YES", "1") ~ 1,
    xc %in% c("NO", "2") ~ 0,
    TRUE ~ NA_real_
  )
}

compute_egfr_2021 <- function(scr_mgdl, age, sex_raw) {
  sex_chr <- toupper(trimws(as.character(sex_raw)))
  sex_num <- suppressWarnings(as.numeric(as.character(sex_raw)))
  male <- case_when(
    sex_num == 1 ~ TRUE,
    sex_num == 2 ~ FALSE,
    sex_chr == "MALE" ~ TRUE,
    sex_chr == "FEMALE" ~ FALSE,
    TRUE ~ NA
  )
  kappa <- ifelse(male, 0.9, 0.7)
  alpha <- ifelse(male, -0.302, -0.241)
  sex_mult <- ifelse(male, 1, 1.012)
  142 * pmin(scr_mgdl / kappa, 1)^alpha * pmax(scr_mgdl / kappa, 1)^(-1.200) * (0.9938^age) * sex_mult
}

compute_acr <- function(ua, ucr, acr_direct = NULL) {
  calc <- ifelse(!is.na(ua) & !is.na(ucr) & ucr > 0, (ua / ucr) * 100, NA_real_)
  dplyr::coalesce(acr_direct, calc)
}

compute_phenoage_components <- function(df, include_scr = TRUE, include_alb = TRUE) {
  # Levine et al. (2018) coefficients expect: albumin g/L, creatinine µmol/L, glucose mmol/L.
  # NHANES provides: albumin g/dL, creatinine mg/dL, glucose often mg/dL (we standardize to mg/dL in pi
  alb_gL   <- df$alb * 10
  scr_umol <- df$scr * 88.4
  glu_mmol <- df$glu / 18.018
  alb_term <- if (include_alb) (-0.0336 * alb_gL) else 0
  scr_term <- if (include_scr) ( 0.0095 * scr_umol) else 0
  glu_term <- 0.1953 * glu_mmol

  xb <- -19.907 +
    alb_term +
    scr_term +
    glu_term +
    0.0954 * log(pmax(df$crp, 1e-6)) -
```

```r
    0.0120 * df$lym +
    0.0268 * df$mcv +
    0.3306 * df$rdw +
    0.00188 * df$alp +
    0.0554 * df$wbc +
    0.0804 * df$age

  xb <- pmin(pmax(xb, -50), 50)
  gamma_const <- (exp(120 * 0.0076927) - 1) / 0.0076927
  hazard <- exp(xb) * gamma_const
  mort <- -expm1(-hazard)
  mort <- pmin(pmax(mort, 1e-12), 1 - 1e-12)
  pheno <- 141.50 + log(-0.00553 * log1p(-mort)) / 0.090165

  list(xb = xb, mort = mort, phenoage = pheno)
}

safe_phenoage <- function(df) {
  compute_phenoage_components(df, include_scr = TRUE, include_alb = TRUE)$phenoage
}

safe_qgcomp_extract <- function(fit_obj, model_label, n_obs) {
  if (is.null(fit_obj)) {
    return(tibble(model = model_label, beta = NA_real_, conf.low = NA_real_, conf.high = NA_real_, p.val
  }
  cf <- coef(fit_obj)
  psi_name <- grep("^psi", names(cf), value = TRUE)[1]
  if (is.na(psi_name)) psi_name <- grep("psi", names(cf), value = TRUE)[1]
  if (is.na(psi_name)) {
    message("qgcomp terms for [", model_label, "]: no psi found in ", paste(names(cf), collapse = ", "))
    return(tibble(model = model_label, beta = NA_real_, conf.low = NA_real_, conf.high = NA_real_, p.val
  }
  beta <- unname(cf[psi_name])
  vc <- tryCatch(vcov(fit_obj), error = function(e) NULL)
  if (!is.null(vc) && psi_name %in% rownames(vc) && psi_name %in% colnames(vc)) {
    se <- sqrt(vc[psi_name, psi_name])
    lo <- beta - 1.96 * se
    hi <- beta + 1.96 * se
    p_val <- 2 * pnorm(-abs(beta / se))
  } else {
    lo <- hi <- p_val <- NA_real_
  }
  tibble(model = model_label, beta = beta, conf.low = lo, conf.high = hi, p.value = p_val, N = n_obs)
}

resolve_weight_var <- function(df, weight_type = "auto", exposure_family = "pfas") {
  cols <- names(df)
  if (weight_type != "auto" && weight_type != "MEC") {
    if (!(weight_type %in% cols)) stop("Requested weight variable not found: ", weight_type)
    return(list(var = weight_type, note = paste("User-specified weight:", weight_type)))
  }
  if ("WTMEC2YR" %in% cols) return(list(var = "WTMEC2YR", note = "Generic fallback: using MEC weight WT
  stop("No supported NHANES weight variable found.")
```

```r
}

apply_lod_rule <- function(df, varname) {
  x <- suppressWarnings(as.numeric(as.character(df[[varname]])))

  base <- sub("^(LBX|URX)", "", varname)
  lc_candidates <- c(
    paste0(varname, "LC"),
    paste0("LBD", base, "LC"),
    paste0("URD", base, "LC"),
    # NHANES often uses *L (not *LC) as the detection-limit flag.
    paste0(varname, "L"),
    paste0("LBD", base, "L"),
    paste0("URD", base, "L")
  )
  lc_candidates <- lc_candidates[lc_candidates %in% names(df)]

  if (length(lc_candidates) > 0) {
    lc <- df[[lc_candidates[1]]]
    lc_num <- suppressWarnings(as.numeric(as.character(lc)))
    lc_chr <- toupper(trimws(as.character(lc)))
    below <- (!is.na(lc_num) & lc_num == 1) | str_detect(lc_chr, "BELOW|<")
    below[is.na(below)] <- FALSE
    # NHANES convention: when LC==1, the concentration column typically holds the LOD value.
    # Standard replacement: value / sqrt(2).
    x[below] <- x[below] / sqrt(2)
    # If value is still NA/non-positive after that, use smallest observed LOD / sqrt(2).
    lod_vals <- x[below & x > 0]
    fallback <- if (length(lod_vals) > 0) min(lod_vals, na.rm = TRUE) else NA_real_
    x[below & (is.na(x) | x <= 0)] <- fallback
    return(x)
  }
  # If no LOD indicator exists, do not impute missing values.
  x
}
```

## Data Acquisition and Cycle Availability

```r
# Cycle-aware: 2003-2004 (C) uses Lxx_ names (L40, L25, L06CRP, L06ALB, L24PFC, L06BMT, L06UHM); 2005+
table_candidates <- list(
  DEMO = c("DEMO"),
  BMX = c("BMX"),
  SMQ = c("SMQ"),
  ALQ = c("ALQ"),
  PAQ = c("PAQ"),
  DIQ = c("DIQ"),
  BPQ = c("BPQ"),
  MCQ = c("MCQ"),
  BIOPRO = c("L40", "BIOPRO"),
  CBC = c("L25", "CBC"),
  CRP = c("L06CRP", "CRP", "HSCRP"),
```

```r
  ALB_CR = c("L06ALB", "ALB_CR"),
  GLU = c("L10", "GLU"),
  PFAS = c("L24PFC", "PFAS", "PFC", "P_PFAS"),
  BLOOD_METALS = c("L06BMT", "PBCD"),
  URINE_METALS = c("L06UHM", "UHM", "UM")
)

download_cycle <- function(cycle, exposure_family = "pfas", verbose = TRUE) {
  core_domains <- c("DEMO", "BMX", "SMQ", "ALQ", "PAQ", "DIQ", "BPQ", "MCQ", "BIOPRO", "CBC", "CRP", "A
  exp_domains <- switch(
    exposure_family,
    pfas = c("PFAS"),
    metals = c("BLOOD_METALS", "URINE_METALS"),
    both = c("PFAS", "BLOOD_METALS", "URINE_METALS"),
    c("PFAS")
  )
  all_domains <- c(core_domains, exp_domains)

  resolved <- lapply(all_domains, function(dom) resolve_table(cycle, table_candidates[[dom]], domain = 
  names(resolved) <- all_domains
  resolved <- resolved[!vapply(resolved, is.null, logical(1))]
  if (!"DEMO" %in% names(resolved)) return(NULL)

  dat <- resolved$DEMO$data
  if (!"SEQN" %in% names(dat)) return(NULL)

  for (nm in setdiff(names(resolved), "DEMO")) {
    d <- resolved[[nm]]$data
    # Rename glucose vars from GLU table before joining so they do not collide with BIOPRO/L40.
    if (nm == "GLU") {
      if ("LBXGLU" %in% names(d)) names(d)[names(d) == "LBXGLU"] <- "LBXGLU_GLU"
      if ("LBXSGL" %in% names(d)) names(d)[names(d) == "LBXSGL"] <- "LBXSGL_GLU"
    }
    if ("SEQN" %in% names(d)) dat <- left_join(dat, d, by = "SEQN")
  }
  # After merge: define explicit glucose sources. Primary analysis uses MEC chemistry glucose for measu
  dat$glu_mec  <- coalesce_num(dat, c("LBXGLU", "LBXSGL"))
  dat$glu_glu  <- coalesce_num(dat, c("LBXGLU_GLU", "LBXSGL_GLU"))
  dat$glu     <- dat$glu_mec
  dat$cycle <- cycle
  # Store provenance for key tables so we can audit weight/source compatibility.
  if ("CRP" %in% names(resolved)) dat$crp_table <- resolved$CRP$table
  if ("GLU" %in% names(resolved)) dat$glu_table <- resolved$GLU$table
  # Warn if joins created .x/.y duplicate columns (would break coalesce_num).
  dup_suffix_cols <- names(dat)[grepl("\\.x$|\\.y$", names(dat))]
  if (length(dup_suffix_cols) > 0) {
    warning("Cycle ", cycle, ": duplicate columns from join: ", paste(dup_suffix_cols, collapse = ", ")
  }
  dat
}

cycle_data <- lapply(config$cycles, download_cycle, exposure_family = config$exposure_family, verbose =
cycle_data <- cycle_data[!vapply(cycle_data, is.null, logical(1))]
```

```r
if (length(cycle_data) == 0) stop("No cycles were successfully downloaded.")

all_cols <- sort(unique(unlist(lapply(cycle_data, names))))
cycle_data <- lapply(cycle_data, function(df) {
  miss <- setdiff(all_cols, names(df))
  if (length(miss) > 0) for (m in miss) df[[m]] <- NA
  df <- df[, all_cols, drop = FALSE]
  df %>% mutate(across(everything(), as.character))
})
nhanes_raw <- bind_rows(cycle_data)

kable(
  data.frame(
    cycle = names(table(config$cycles))[names(table(config$cycles)) %in% unique(nhanes_raw$cycle)],
    n = as.integer(table(nhanes_raw$cycle))
  ),
  caption = "Cycle availability and downloaded participant counts"
)
```

Table 1: Cycle availability and downloaded participant counts

| cycle | n |
|---|---|
| 2003-2004 | 10122 |
| 2005-2006 | 10348 |
| 2007-2008 | 10149 |
| 2009-2010 | 10537 |
| 2011-2012 | 9756 |
| 2013-2014 | 10175 |
| 2015-2016 | 9971 |
| 2017-2018 | 9254 |

## Variable Construction: PhenoAge, CKD, Covariates

```r
dpq_vars <- names(nhanes_raw)[str_detect(names(nhanes_raw), "^DPQ0")]

dat <- nhanes_raw %>%
  mutate(across(any_of(dpq_vars), parse_phq_item)) %>%
  mutate(
    age = suppressWarnings(as.numeric(as.character(RIDAGEYR))),
    sex = case_when(
      suppressWarnings(as.numeric(as.character(RIAGENDR))) == 1 ~ "Male",
      suppressWarnings(as.numeric(as.character(RIAGENDR))) == 2 ~ "Female",
      toupper(trimws(as.character(RIAGENDR))) == "MALE" ~ "Male",
      toupper(trimws(as.character(RIAGENDR))) == "FEMALE" ~ "Female",
      TRUE ~ NA_character_
    ),
    race_eth = {
      r3 <- suppressWarnings(as.numeric(as.character(RIDRETH3)))
      r1 <- suppressWarnings(as.numeric(as.character(RIDRETH1)))
```

```r
      r3_chr <- toupper(trimws(as.character(RIDRETH3)))
      r1_chr <- toupper(trimws(as.character(RIDRETH1)))
      case_when(
        !is.na(r3) & r3 %in% c(1, 2) ~ "Hispanic",
        !is.na(r3) & r3 == 3 ~ "Non-Hispanic White",
        !is.na(r3) & r3 == 4 ~ "Non-Hispanic Black",
        !is.na(r3) & r3 == 6 ~ "Non-Hispanic Asian",
        !is.na(r3) & r3 %in% c(5, 7) ~ "Other/Multi",
        is.na(r3) & r1 %in% c(1, 2) ~ "Hispanic",
        is.na(r3) & r1 == 3 ~ "Non-Hispanic White",
        is.na(r3) & r1 == 4 ~ "Non-Hispanic Black",
        is.na(r3) & r1 == 5 ~ "Other/Multi",
        str_detect(r3_chr, "NON-HISPANIC WHITE") ~ "Non-Hispanic White",
        str_detect(r3_chr, "NON-HISPANIC BLACK") ~ "Non-Hispanic Black",
        str_detect(r3_chr, "NON-HISPANIC ASIAN") ~ "Non-Hispanic Asian",
        str_detect(r3_chr, "OTHER RACE|MULTI") ~ "Other/Multi",
        str_detect(r3_chr, "^MEXICAN AMERICAN$|^OTHER HISPANIC$|^HISPANIC$") ~ "Hispanic",
        str_detect(r1_chr, "NON-HISPANIC WHITE") ~ "Non-Hispanic White",
        str_detect(r1_chr, "NON-HISPANIC BLACK") ~ "Non-Hispanic Black",
        str_detect(r1_chr, "OTHER RACE|MULTI") ~ "Other/Multi",
        str_detect(r1_chr, "^MEXICAN AMERICAN$|^OTHER HISPANIC$|^HISPANIC$") ~ "Hispanic",
        TRUE ~ NA_character_
      )
    },
    education = {
      e_num <- suppressWarnings(as.numeric(as.character(DMDEDUC2)))
      e_chr <- toupper(trimws(as.character(DMDEDUC2)))
      case_when(
        e_num %in% c(1, 2) ~ "<HS",
        e_num == 3 ~ "HS/GED",
        e_num %in% c(4, 5) ~ ">HS",
        str_detect(e_chr, "LESS THAN 9TH|9TH.*12TH") ~ "<HS",
        str_detect(e_chr, "HIGH SCHOOL|GED") ~ "HS/GED",
        str_detect(e_chr, "SOME COLLEGE|COLLEGE GRADUATE") ~ ">HS",
        TRUE ~ NA_character_
      )
    },
    pir = suppressWarnings(as.numeric(as.character(INDFMPIR))),
    bmi = suppressWarnings(as.numeric(as.character(BMXBMI))),
    smoker = {
      smq040_chr <- toupper(trimws(as.character(SMQ040)))
      smq040_num <- suppressWarnings(as.numeric(as.character(SMQ040)))
      case_when(
        smq040_num %in% c(1, 2) ~ 1,
        smq040_num == 3 ~ 0,
        str_detect(smq040_chr, "EVERY DAY|SOME DAYS") ~ 1,
        str_detect(smq040_chr, "^NOT AT ALL$") ~ 0,
        TRUE ~ NA_real_
      )
    },
    alcohol = parse_yes_no(ALQ151),
    phys_active = case_when(
      parse_yes_no(PAQ650) == 1 | parse_yes_no(PAQ665) == 1 ~ 1,
```

```r
      parse_yes_no(PAQ650) == 0 & parse_yes_no(PAQ665) == 0 ~ 0,
      TRUE ~ NA_real_
    ),
    diabetes = yesno01(DIQ010),
    hypertension = yesno01(BPQ020),
    cvd = case_when(
      parse_yes_no(MCQ160B) == 1 | parse_yes_no(MCQ160C) == 1 | parse_yes_no(MCQ160E) == 1 | parse_yes_
      parse_yes_no(MCQ160B) == 0 & parse_yes_no(MCQ160C) == 0 & parse_yes_no(MCQ160E) == 0 & parse_yes_
      TRUE ~ NA_real_
    ),
    phenoage_accel = NA_real_
  )

# Compute biomarkers, CKD markers, and design variables outside mutate (avoids cur_data_all()).
dat$alb <- coalesce_num(dat, c("LBXSAL"))
dat$scr <- coalesce_num(dat, c("LBXSCR", "LBDSCRSI"))
# Glucose: glu_mec / glu_glu / glu set in download_cycle (primary uses glu_mec by design).
if (!"glu" %in% names(dat)) dat$glu <- dat$glu_mec
dat$crp <- coalesce_num(dat, c("LBXCRP", "LBXHSCRP"))
dat$lym <- coalesce_num(dat, c("LBXLYPCT"))
dat$mcv <- coalesce_num(dat, c("LBXMCV", "LBXMCVSI"))
dat$rdw <- coalesce_num(dat, c("LBXRW", "LBXRDW"))
dat$alp <- coalesce_num(dat, c("LBXAPSI", "LBXSAPSI"))
dat$wbc <- coalesce_num(dat, c("LBXWBCSI", "LBXWBC"))

dat$ualb <- coalesce_num(dat, c("URXUMA"))
dat$ucr <- coalesce_num(dat, c("URXUCR", "URDUCR"))
dat$acr_direct <- coalesce_num(dat, c("URDACT"))
dat$acr <- compute_acr(dat$ualb, dat$ucr, dat$acr_direct)
dat$egfr <- compute_egfr_2021(dat$scr, dat$age, dat$RIAGENDR)
dat$ckd_egfr <- ifelse(!is.na(dat$egfr), as.integer(dat$egfr < 60), NA_integer_)
dat$ckd_acr <- ifelse(!is.na(dat$acr), as.integer(dat$acr >= 30), NA_integer_)
dat$ckd <- case_when(
  config$ckd_def == "egfr_or_acr" & (dat$ckd_egfr == 1 | dat$ckd_acr == 1) ~ 1L,
  config$ckd_def == "egfr_or_acr" & (!is.na(dat$ckd_egfr) | !is.na(dat$ckd_acr)) ~ 0L,
  config$ckd_def == "egfr_only" ~ dat$ckd_egfr,
  config$ckd_def == "acr_only" ~ dat$ckd_acr,
  TRUE ~ NA_integer_
)
dat$ckd_stage <- cut(
  dat$egfr, breaks = c(-Inf, 15, 30, 45, 60, 90, Inf),
  labels = c("G5", "G4", "G3b", "G3a", "G2", "G1"),
  right = FALSE  # KDIGO: left-closed intervals [lo, hi). E.g., 60 -> G2, 59.9 -> G3a.
)
dat$strata <- coalesce_num(dat, c("SDMVSTRA"))
dat$psu <- coalesce_num(dat, c("SDMVPSU"))

# Coding audits (raw -> derived) to prevent silent recode failures.
kable(
  dat %>% count(cycle, ridreth3_missing = is.na(RIDRETH3)) %>% arrange(cycle, ridreth3_missing),
  caption = "Race audit: availability of RIDRETH3 by cycle"
)
```

Table 2: Race audit: availability of RIDRETH3 by cycle

| cycle | ridreth3_missing | n |
|---|---|---|
| 2003-2004 | TRUE | 10122 |
| 2005-2006 | TRUE | 10348 |
| 2007-2008 | TRUE | 10149 |
| 2009-2010 | TRUE | 10537 |
| 2011-2012 | FALSE | 9756 |
| 2013-2014 | FALSE | 10175 |
| 2015-2016 | FALSE | 9971 |
| 2017-2018 | FALSE | 9254 |

```
kable(as.data.frame(table(dat$RIDRETH1, useNA = "ifany")), caption = "Raw RIDRETH1 distribution")
```

Table 3: Raw RIDRETH1 distribution

| Var1 | Freq |
|---|---|
| Mexican American | 16280 |
| Non-Hispanic Black | 18735 |
| Non-Hispanic White | 29459 |
| Other Hispanic | 7188 |
| Other Race - Including Multi-Racial | 8650 |

```
kable(as.data.frame(table(dat$RIDRETH3, useNA = "ifany")), caption = "Raw RIDRETH3 distribution")
```

Table 4: Raw RIDRETH3 distribution

| Var1 | Freq |
|---|---|
| Mexican American | 6373 |
| Non-Hispanic Asian | 4566 |
| Non-Hispanic Black | 9194 |
| Non-Hispanic White | 12863 |
| Other Hispanic | 4164 |
| Other Race - Including Multi-Racial | 1996 |
| NA | 41156 |

```
kable(as.data.frame(table(dat$race_eth, useNA = "ifany")), caption = "Derived race_eth distribution")
```

Table 5: Derived race_eth distribution

| Var1 | Freq |
|---|---|
| Hispanic | 23468 |
| Non-Hispanic Asian | 4566 |
| Non-Hispanic Black | 18735 |
| Non-Hispanic White | 29459 |
| Other/Multi | 4084 |

```r
kable(as.data.frame(table(dat$SMQ040, useNA = "ifany")), caption = "Raw SMQ040 distribution")
```

Table 6: Raw SMQ040 distribution

| Var1 | Freq |
|---|---|
| Every day | 7593 |
| Not at all | 10892 |
| Refused | 3 |
| Some days | 1753 |
| NA | 60071 |

```r
kable(as.data.frame(table(dat$DIQ010, useNA = "ifany")), caption = "Raw DIQ010 distribution")
```

Table 7: Raw DIQ010 distribution

| Var1 | Freq |
|---|---|
| Borderline | 1062 |
| Don't know | 48 |
| No | 69943 |
| Refused | 3 |
| Yes | 5790 |
| NA | 3466 |

```r
kable(as.data.frame(table(dat$BPQ020, useNA = "ifany")), caption = "Raw BPQ020 distribution")
```

Table 8: Raw BPQ020 distribution

| Var1 | Freq |
|---|---|
| Don't know | 83 |
| No | 34534 |
| Yes | 16221 |
| NA | 29474 |

```r
kable(as.data.frame(table(dat$smoker, useNA = "ifany")), caption = "Derived smoker distribution")
```

Table 9: Derived smoker distribution

| Var1 | Freq |
|---|---|
| 0 | 10892 |
| 1 | 9346 |
| NA | 60074 |

```r
kable(as.data.frame(table(dat$diabetes, useNA = "ifany")), caption = "Derived diabetes distribution")
```

Table 10: Derived diabetes distribution

| Var1 | Freq |
|---|---|
| 0 | 69943 |
| 1 | 5790 |
| NA | 4579 |

```
kable(as.data.frame(table(dat$hypertension, useNA = "ifany")), caption = "Derived hypertension distribu
```

Table 11: Derived hypertension distribution

| Var1 | Freq |
|---|---|
| 0 | 34534 |
| 1 | 16221 |
| NA | 29557 |

```
# Codebook-aligned harmonization checks (raw coding support by cycle).
check_raw_levels <- function(df, var, allowed_patterns) {
  if (!(var %in% names(df))) return(tibble(variable = var, cycle = NA_character_, n_unexpected = NA_int
  df %>%
    mutate(.raw = toupper(trimws(as.character(.data[[var]])))) %>%
    group_by(cycle) %>%
    summarise(
      n_nonmissing = sum(!is.na(.raw) & .raw != ""),
      n_unexpected = sum(!is.na(.raw) & .raw != "" & !str_detect(.raw, allowed_patterns)),
      unexpected_examples = paste(head(unique(.raw[!is.na(.raw) & .raw != "" & !str_detect(.raw, allowe
      .groups = "drop"
    ) %>%
    mutate(variable = var) %>%
    select(variable, cycle, n_nonmissing, n_unexpected, unexpected_examples)
}

harmonization_checks <- bind_rows(
  check_raw_levels(dat, "RIAGENDR", "^(1|2|MALE|FEMALE)$"),
  check_raw_levels(dat, "DMDEDUC2", "^(1|2|3|4|5|7|9)$|LESS THAN 9TH|9-11TH|HIGH SCHOOL|GED|SOME COLLEG
  check_raw_levels(dat, "SMQ040", "^(1|2|3|7|9)$|EVERY DAY|SOME DAYS|NOT AT ALL|REFUSED|DON'T KNOW|DONT
  check_raw_levels(dat, "DIQ010", "^(1|2|3|7|9)$|YES|NO|BORDERLINE|REFUSED|DON'T KNOW|DONT KNOW"),
  check_raw_levels(dat, "BPQ020", "^(1|2|7|9)$|YES|NO|REFUSED|DON'T KNOW|DONT KNOW")
)
kable(harmonization_checks, caption = "Codebook-aligned raw coding checks by cycle. Non-zero unexpected
```

Table 12: Codebook-aligned raw coding checks by cycle. Non-zero unexpected counts indicate coding values not covered by harmonization rules.

| variable | cycle | n_nonmissing | n_unexpected | unexpected_examples |
|---|---|---|---|---|
| RIAGENDR | 2003-2004 | 10122 | 0 | |
| RIAGENDR | 2005-2006 | 10348 | 0 | |
| RIAGENDR | 2007-2008 | 10149 | 0 | |

| variable | cycle | n_nonmissing | n_unexpected | unexpected_examples |
|---|---|---|---|---|
| RIAGENDR | 2009-2010 | 10537 | 0 | |
| RIAGENDR | 2011-2012 | 9756 | 0 | |
| RIAGENDR | 2013-2014 | 10175 | 0 | |
| RIAGENDR | 2015-2016 | 9971 | 0 | |
| RIAGENDR | 2017-2018 | 9254 | 0 | |
| DMDEDUC2 | 2003-2004 | 5041 | 0 | |
| DMDEDUC2 | 2005-2006 | 4979 | 0 | |
| DMDEDUC2 | 2007-2008 | 5935 | 0 | |
| DMDEDUC2 | 2009-2010 | 6218 | 0 | |
| DMDEDUC2 | 2011-2012 | 5560 | 0 | |
| DMDEDUC2 | 2013-2014 | 5769 | 0 | |
| DMDEDUC2 | 2015-2016 | 5719 | 0 | |
| DMDEDUC2 | 2017-2018 | 5569 | 0 | |
| SMQ040 | 2003-2004 | 2496 | 0 | |
| SMQ040 | 2005-2006 | 2348 | 0 | |
| SMQ040 | 2007-2008 | 2802 | 0 | |
| SMQ040 | 2009-2010 | 2866 | 0 | |
| SMQ040 | 2011-2012 | 2369 | 0 | |
| SMQ040 | 2013-2014 | 2579 | 0 | |
| SMQ040 | 2015-2016 | 2422 | 0 | |
| SMQ040 | 2017-2018 | 2359 | 0 | |
| DIQ010 | 2003-2004 | 9645 | 0 | |
| DIQ010 | 2005-2006 | 9822 | 0 | |
| DIQ010 | 2007-2008 | 9666 | 0 | |
| DIQ010 | 2009-2010 | 10109 | 0 | |
| DIQ010 | 2011-2012 | 9363 | 0 | |
| DIQ010 | 2013-2014 | 9769 | 0 | |
| DIQ010 | 2015-2016 | 9575 | 0 | |
| DIQ010 | 2017-2018 | 8897 | 0 | |
| BPQ020 | 2003-2004 | 6138 | 0 | |
| BPQ020 | 2005-2006 | 6139 | 0 | |
| BPQ020 | 2007-2008 | 6546 | 0 | |
| BPQ020 | 2009-2010 | 6889 | 0 | |
| BPQ020 | 2011-2012 | 6174 | 0 | |
| BPQ020 | 2013-2014 | 6464 | 0 | |
| BPQ020 | 2015-2016 | 6327 | 0 | |
| BPQ020 | 2017-2018 | 6161 | 0 | |

```r
# --- CRP unit detection and conversion ---
# PhenoAge (Levine 2018) uses CRP in mg/dL. NHANES LBXCRP is typically mg/dL;
# LBXHSCRP or CRP tables in some cycles may report mg/L.
# Step 1: check variable/table provenance.
crp_by_cycle <- dat %>%
  group_by(cycle) %>%
  summarise(
    source_table = if ("crp_table" %in% names(pick(everything()))) paste(unique(na.omit(crp_table)), co
    n_obs = sum(!is.na(crp)),
    median_crp = median(crp, na.rm = TRUE),
    p75 = quantile(crp, 0.75, na.rm = TRUE),
    p90 = quantile(crp, 0.90, na.rm = TRUE),
    .groups = "drop"
  )
```

```r
kable(crp_by_cycle, caption = "CRP diagnostic by cycle -- verify units (PhenoAge expects mg/dL). If sou
```

Table 13: CRP diagnostic by cycle – verify units (PhenoAge expects mg/dL). If source table is HSCRP, values may be mg/L.

| cycle | source_table | n_obs | median_crp | p75 | p90 |
|---|---|---|---|---|---|
| 2003-2004 | | 0 | NA | NA | NA |
| 2005-2006 | CRP_D | 8172 | 0.120 | 0.38 | 0.890 |
| 2007-2008 | CRP_E | 6809 | 0.160 | 0.40 | 0.920 |
| 2009-2010 | CRP_F | 8299 | 0.130 | 0.35 | 0.830 |
| 2011-2012 | | 0 | NA | NA | NA |
| 2013-2014 | | 0 | NA | NA | NA |
| 2015-2016 | HSCRP_I | 7867 | 1.200 | 3.60 | 8.200 |
| 2017-2018 | HSCRP_J | 7250 | 1.355 | 3.59 | 7.462 |

```r
# Step 2: provenance-based detection (strongest signal).
crp_tables_used <- if ("crp_table" %in% names(dat)) unique(na.omit(dat$crp_table)) else character(0)
provenance_suggests_mgl <- length(crp_tables_used) > 0 && any(str_detect(crp_tables_used, regex("HSCRP"
# Step 3: distribution-based backup -- require median > 1.5 AND p75 > 3 to trigger conversion.
crp_overall_med <- median(dat$crp, na.rm = TRUE)
crp_overall_p75 <- quantile(dat$crp, 0.75, na.rm = TRUE)
distribution_suggests_mgl <- !is.na(crp_overall_med) && crp_overall_med > 1.5 &&
                             !is.na(crp_overall_p75) && crp_overall_p75 > 3

if (provenance_suggests_mgl || distribution_suggests_mgl) {
  reason <- if (provenance_suggests_mgl) {
    paste0("CRP source table(s) [", paste(crp_tables_used, collapse = ", "), "] suggest mg/L.")
  } else {
    paste0("Distribution heuristic: median=", round(crp_overall_med, 3), ", p75=", round(crp_overall_p7
  }
  message(">>> Converting CRP from mg/L to mg/dL (/ 10). Reason: ", reason)
  dat$crp <- dat$crp / 10
} else {
  message(">>> CRP assumed to be in mg/dL (median=", round(crp_overall_med, 4),
          ", p75=", round(crp_overall_p75, 4), "). Verify against NHANES documentation.")
}

# --- Glucose provenance diagnostic ---
# Coerce glu to numeric (nhanes_raw has character columns from bind_rows); needed for summarise and lat
dat$glu <- suppressWarnings(as.numeric(as.character(dat$glu)))
glu_by_cycle <- dat %>%
  group_by(cycle) %>%
  summarise(
    source_table = if ("glu_table" %in% names(pick(everything()))) paste(unique(na.omit(glu_table)), co
    n_glu = sum(!is.na(glu)),
    median_glu = median(glu, na.rm = TRUE),
    min_glu = min(glu, na.rm = TRUE),
    max_glu = max(glu, na.rm = TRUE),
    .groups = "drop"
  )
kable(glu_by_cycle, caption = "Glucose provenance by cycle. If source_table is a fasting component (GLU
```

Table 14: Glucose provenance by cycle. If source_table is a fasting component (GLU_*), use a compatible fasting subsample weight.

| cycle | source_table | n_glu | median_glu | min_glu | max_glu |
|---|---|---|---|---|---|
| 2003-2004 | L10_C | 6492 | 89 | 46 | 512 |
| 2005-2006 | GLU_D | 6434 | 90 | 42 | 513 |
| 2007-2008 | GLU_E | 6377 | 91 | 33 | 559 |
| 2009-2010 | GLU_F | 6860 | 91 | 34 | 777 |
| 2011-2012 | GLU_G | 5976 | 91 | 31 | 617 |
| 2013-2014 | GLU_H | 6553 | 93 | 49 | 577 |
| 2015-2016 | GLU_I | 6257 | 92 | 19 | 610 |
| 2017-2018 | GLU_J | 5901 | 92 | 47 | 626 |

```
glu_med <- median(dat$glu, na.rm = TRUE)
if (!is.na(glu_med) && glu_med < 6) {
  warning("Glucose median is < 6 -- this looks like mmol/L. Converting to mg/dL ( x 18.018).")
  dat$glu <- dat$glu * 18.018
}

# Warn based on stored table provenance, not just weight variable existence.
glu_tables_used <- if ("glu_table" %in% names(dat)) unique(na.omit(dat$glu_table)) else character(0)
glu_is_fasting <- length(glu_tables_used) > 0 && any(str_detect(glu_tables_used, regex("^GLU_", ignore_
if (glu_is_fasting) {
  message(">>> IMPORTANT: Glucose sourced from fasting component table(s) [",
          paste(glu_tables_used, collapse = ", "),
          "]. A compatible fasting subsample weight (e.g., WTSAF2YR) is required. ",
          "Using a non-fasting or otherwise incompatible weight can bias estimates. ",
          "Review the weight audit; override config$weight_type only if auto-selection does not resolve
} else if (length(glu_tables_used) > 0) {
  message(">>> Glucose source table(s): ", paste(glu_tables_used, collapse = ", "),
          ". Verify whether this is a fasting component.")
}

# ------------------------------------------------------------
# CLEAN SPECIAL NHANES MISSING CODES FOR LAB VARIABLES
# ------------------------------------------------------------
clean_nhanes_numeric <- function(x) {
  x <- suppressWarnings(as.numeric(as.character(x)))
  special_codes <- c(
    7, 77, 777, 7777, 77777,
    9, 99, 999, 9999, 99999,
    888, 8888, 88888
  )
  x[x %in% special_codes] <- NA_real_
  x
}

dat$alb <- clean_nhanes_numeric(dat$alb)
dat$scr <- clean_nhanes_numeric(dat$scr)
dat$glu <- clean_nhanes_numeric(dat$glu)
dat$crp <- clean_nhanes_numeric(dat$crp)
dat$lym <- clean_nhanes_numeric(dat$lym)
dat$mcv <- clean_nhanes_numeric(dat$mcv)
```

```r
dat$rdw <- clean_nhanes_numeric(dat$rdw)
dat$alp <- clean_nhanes_numeric(dat$alp)
dat$wbc <- clean_nhanes_numeric(dat$wbc)

# ------------------------------------------------------------
# HARD PLAUSIBILITY CHECK (retain CRP >1 mg/dL in primary; flag as acute)
# ------------------------------------------------------------
dat <- dat %>%
  mutate(
    crp_acute = ifelse(!is.na(crp) & crp > 1, 1L, 0L),
    wbc = ifelse(wbc > 50, NA_real_, wbc),
    glu = ifelse(glu > 600, NA_real_, glu),
    alp = ifelse(alp > 500, NA_real_, alp)
  )

# --- General biomarker plausibility ---
biomarker_ranges <- list(
  alb  = c(1, 7,    "g/dL"),
  scr  = c(0.1, 25, "mg/dL"),
  glu  = c(20, 600,  "mg/dL"),
  crp  = c(0.001, 3, "mg/dL (post-conversion if applied)"),
  lym  = c(1, 80,    "%"),
  mcv  = c(50, 150,  "fL"),
  rdw  = c(8, 30,    "%"),
  alp  = c(5, 500,   "U/L"),
  wbc  = c(1, 50,    "1000 cells/uL")
)
biomarker_check <- lapply(names(biomarker_ranges), function(nm) {
  vals <- dat[[nm]]
  rng  <- biomarker_ranges[[nm]]
  lo   <- as.numeric(rng[1]); hi <- as.numeric(rng[2]); unit <- rng[3]
  v    <- vals[!is.na(vals)]
  n_out <- sum(v < lo | v > hi)
  tibble(biomarker = nm, unit = unit, n_obs = length(v),
         min = if (length(v) > 0) min(v) else NA_real_,
         p25 = if (length(v) > 0) quantile(v, 0.25) else NA_real_,
         median = if (length(v) > 0) median(v) else NA_real_,
         p75 = if (length(v) > 0) quantile(v, 0.75) else NA_real_,
         max = if (length(v) > 0) max(v) else NA_real_,
         expected_lo = lo, expected_hi = hi,
         n_out_of_range = n_out)
}) %>% bind_rows()
kable(biomarker_check, caption = "PhenoAge biomarker ranges after unit conversion (verify units)")
```

Table 15: PhenoAge biomarker ranges after unit conversion (verify units)

| biomarker | unit | n_obs | min | p25 | median | p75 | max | expected_lo | expected_hi | n_out_of_range |
|---|---|---|---|---|---|---|---|---|---|---|
| alb | g/dL | 50853 | 1.200 | 4.000 | 4.300 | 4.500 | 5.60 | 1e+00 | 7 | 0 |
| scr | mg/dL | 50848 | 0.140 | 0.700 | 0.820 | 0.990 | 17.80 | 1e-01 | 25 | 0 |
| glu | mg/dL | 49165 | 19.000 | 84.000 | 91.000 | 102.000 | 577.00 | 2e+01 | 600 | 1 |

| biomarker | unit | n_obs | min | p25 | median | p75 | max | expected_lo | expected_hi | out_of_range |
|---|---|---|---|---|---|---|---|---|---|---|
| crp | mg/dL (post-conversion if applied) | 38397 | 0.001 | 0.008 | 0.032 | 0.116 | 18.85 | 1e-03 | 3 | 197 |
| lym | % | 65864 | 2.600 | 26.300 | 32.500 | 39.900 | 94.50 | 1e+00 | 80 | 21 |
| mcv | fL | 65903 | 35.400 | 83.600 | 87.700 | 91.400 | 125.30 | 5e+01 | 150 | 1 |
| rdw | % | 65992 | 6.300 | 12.400 | 12.900 | 13.600 | 37.80 | 8e+00 | 30 | 7 |
| alp | U/L | 49756 | 10.000 | 56.000 | 70.000 | 90.000 | 500.00 | 5e+00 | 500 | 0 |
| wbc | 1000 cells/uL | 64614 | 1.400 | 5.700 | 6.900 | 8.400 | 44.80 | 1e+00 | 50 | 0 |

```r
for (nm in names(biomarker_ranges)) {
  v <- dat[[nm]][!is.na(dat[[nm]])]
  lo <- as.numeric(biomarker_ranges[[nm]][1])
  hi <- as.numeric(biomarker_ranges[[nm]][2])
  pct_out <- if (length(v) > 0) mean(v < lo | v > hi) else 0
  if (pct_out > 0.20) warning(sprintf(
    "Biomarker '%s': %.1f%% of values outside plausible range [%s, %s]. Check units.",
    nm, pct_out * 100, lo, hi
  ))
}

# Recompute PhenoAge after any unit conversions (CRP mg/L->mg/dL, glucose mmol/L->mg/dL).
pheno_core <- compute_phenoage_components(dat, include_scr = TRUE, include_alb = TRUE)
dat$phenoage <- pheno_core$phenoage

message(
  "PhenoAge diagnostics -- xb median=", round(stats::median(pheno_core$xb, na.rm = TRUE), 3),
  ", mort median=", format(stats::median(pheno_core$mort, na.rm = TRUE), scientific = TRUE, digits = 3)
  ", pct(mort>0.9999)=", round(mean(pheno_core$mort > 0.9999, na.rm = TRUE) * 100, 1), "%",
  ", PhenoAge SD=", round(stats::sd(dat$phenoage, na.rm = TRUE), 3)
)
ph_q <- stats::quantile(dat$phenoage, probs = c(0, 0.25, 0.5, 0.75, 1), na.rm = TRUE)
message(
  "PhenoAge five-number summary -- min=", round(ph_q[[1]], 3),
  ", p25=", round(ph_q[[2]], 3),
  ", median=", round(ph_q[[3]], 3),
  ", p75=", round(ph_q[[4]], 3),
  ", max=", round(ph_q[[5]], 3)
)
if (mean(pheno_core$mort > 0.9999, na.rm = TRUE) > 0.25) {
  warning("More than 25% of participants have mort > 0.9999 after stabilization. Re-check biomarker uni
}
```

## Exposure Processing and LOD Handling

```r
pfas_vars <- c("LBXPFOS", "LBXPFOA", "LBXPFNA", "LBXPFHS", "LBXPFDA")

# --- ISOMER HARMONIZATION (2013-2018) ---
# NHANES split PFOS and PFOA into linear and branched isomers.
# We must apply the LOD/sqrt(2) rule to the individual isomers FIRST, then sum them.
```

```r
isomer_map <- list(
  pfos_lin = c("SSNPFOS", "LBXNFOS"),
  pfos_br  = c("SSMPFOS", "LBXMFOS"),
  pfoa_lin = c("SSNPFOA", "LBXNFOA"),
  pfoa_br  = c("SSBPFOA", "LBXBFOA")
)

lc_map <- list(
  pfos_lin = c("SDNPFOSL", "LBDNFOSL"),
  pfos_br  = c("SDMPFOSL", "LBDMFOSL"),
  pfoa_lin = c("SDNPFOAL", "LBDNFOAL"),
  pfoa_br  = c("SDBPFOAL", "LBDBFOAL")
)

for (chem in c("pfos", "pfoa")) {
  # Get available variables for the cycle
  lin_vars <- isomer_map[[paste0(chem, "_lin")]]
  br_vars  <- isomer_map[[paste0(chem, "_br")]]
  lin_lc   <- lc_map[[paste0(chem, "_lin")]]
  br_lc    <- lc_map[[paste0(chem, "_br")]]

  lin_val <- coalesce_num(dat, lin_vars)
  br_val  <- coalesce_num(dat, br_vars)
  lin_lc_val <- coalesce_num(dat, lin_lc)
  br_lc_val  <- coalesce_num(dat, br_lc)

  # Apply LOD rule: if indicator == 1, divide by sqrt(2)
  lin_val <- ifelse(!is.na(lin_lc_val) & lin_lc_val == 1, lin_val / sqrt(2), lin_val)
  br_val  <- ifelse(!is.na(br_lc_val) & br_lc_val == 1, br_val / sqrt(2), br_val)

  # Sum them to recreate the total concentration
  summed_val <- lin_val + br_val

  # Inject back into the legacy total variables (LBXPFOS or LBXPFOA)
  target_var <- ifelse(chem == "pfos", "LBXPFOS", "LBXPFOA")

  if (target_var %in% names(dat)) {
    dat[[target_var]] <- coalesce_num(dat, c(target_var))
    dat[[target_var]] <- ifelse(is.na(dat[[target_var]]) & !is.na(summed_val), summed_val, dat[[target_
  } else {
    dat[[target_var]] <- summed_val
  }
}
# --- END ISOMER HARMONIZATION ---

blood_metal_vars <- c("LBXBPB", "LBXBCD", "LBXTHG")
urine_metal_vars <- c("URXUAS", "URXUCD", "URXUPB")

chosen_exposures <- switch(
  config$exposure_family,
  pfas = pfas_vars,
  metals = c(blood_metal_vars, urine_metal_vars),
  both = c(pfas_vars, blood_metal_vars, urine_metal_vars),
```

```r
  pfas_vars
)
chosen_exposures <- chosen_exposures[chosen_exposures %in% names(dat)]
if (length(chosen_exposures) < 3) stop("Too few exposure variables found; choose different exposure fam

for (v in chosen_exposures) {
  vv <- apply_lod_rule(dat, v)
  dat[[paste0(v, "_log2")]] <- log2(vv)
}

if (any(urine_metal_vars %in% chosen_exposures)) {
  for (uv in intersect(urine_metal_vars, chosen_exposures)) {
    vv <- apply_lod_rule(dat, uv)
    dat[[paste0(uv, "_ucr_adj_log2")]] <- log2((vv / pmax(dat$ucr, 1e-6)) * 100)
  }
}

exposure_vars <- names(dat)[str_detect(names(dat), "_log2$|_ucr_adj_log2$")]
exposure_vars <- unique(exposure_vars)
# Prevent collinear double-counting: use creatinine-adjusted urine metals only, drop raw _log2 for them
raw_urine_vars <- paste0(urine_metal_vars, "_log2")
exposure_vars <- setdiff(exposure_vars, raw_urine_vars)
# Primary analysis: fixed 4-PFAS subset common across all retained cycles. Sensitivity: available-expos
primary_exposure_vars <- c("LBXPFOS_log2", "LBXPFOA_log2", "LBXPFNA_log2", "LBXPFHS_log2")
primary_exposure_vars <- primary_exposure_vars[primary_exposure_vars %in% exposure_vars]
adaptive_exposure_vars <- exposure_vars

# LOD diagnostic table -- computed on dat (post-merge, pre-log transform) stratified by cycle.
lod_diag <- lapply(chosen_exposures, function(v) {
  x_raw <- suppressWarnings(as.numeric(as.character(dat[[v]])))
  base <- sub("^(LBX|URX)", "", v)
  lc_candidates <- c(
    paste0(v, "LC"), paste0("LBD", base, "LC"), paste0("URD", base, "LC"),
    paste0(v, "L"),  paste0("LBD", base, "L"),  paste0("URD", base, "L")
  )
  lc_var <- lc_candidates[lc_candidates %in% names(dat)]
  has_lc <- length(lc_var) > 0
  if (has_lc) {
    lc_vals <- suppressWarnings(as.numeric(as.character(dat[[lc_var[1]]])))
    below <- !is.na(lc_vals) & lc_vals == 1
  } else {
    below <- rep(FALSE, length(x_raw))
  }

  by_cycle <- dat %>%
    mutate(.x = x_raw, .below = below) %>%
    group_by(cycle) %>%
    summarise(
      n = sum(!is.na(.x)),
      n_below = sum(.below, na.rm = TRUE),
      pct_below = if (sum(!is.na(.x)) > 0) round(sum(.below, na.rm = TRUE) / sum(!is.na(.x)) * 100, 1) 
      median_below = if (sum(.below & !is.na(.x)) > 0) median(.x[.below], na.rm = TRUE) else NA_real_,
      median_above = if (sum(!.below & !is.na(.x)) > 0) median(.x[!.below & !is.na(.x)], na.rm = TRUE) 
```

```
      min_positive = if (any(.x > 0, na.rm = TRUE)) min(.x[.x > 0], na.rm = TRUE) else NA_real_,
      .groups = "drop"
    ) %>%
    mutate(exposure = v, lc_indicator = if (has_lc) lc_var[1] else "none") %>%
    select(exposure, cycle, lc_indicator, everything())
  by_cycle
}) %>% bind_rows()
kable(lod_diag, caption = "LOD diagnostic per exposure by cycle. Verify: median_below ~ LOD value; if i
```

Table 16: LOD diagnostic per exposure by cycle. Verify: median_below ~ LOD value; if it diverges from min_positive, review the LC->x/sqrt(2) assumption.

| exposure | cycle | lc_indicator | n | n_below | pct_below | median_below | median_above | min_positive |
|---|---|---|---|---|---|---|---|---|
| LBXPFOS | 2003-2004 | LBDPFOSL | 2094 | 0 | 0 | NA | 19.900 | 0.300 |
| LBXPFOS | 2005-2006 | LBDPFOSL | 2120 | 0 | 0 | NA | 15.900 | 0.140 |
| LBXPFOS | 2007-2008 | LBDPFOSL | 2100 | 0 | 0 | NA | 13.400 | 0.140 |
| LBXPFOS | 2009-2010 | LBDPFOSL | 2233 | 0 | 0 | NA | 9.000 | 0.140 |
| LBXPFOS | 2011-2012 | LBDPFOSL | 1904 | 0 | 0 | NA | 6.425 | 0.140 |
| LBXPFOS | 2013-2014 | LBDPFOSL | 0 | 0 | NA | NA | NA | NA |
| LBXPFOS | 2015-2016 | LBDPFOSL | 1993 | 0 | 0 | NA | 4.700 | 0.140 |
| LBXPFOS | 2017-2018 | LBDPFOSL | 1929 | 0 | 0 | NA | 4.200 | 0.140 |
| LBXPFOA | 2003-2004 | LBDPFOAL | 2094 | 0 | 0 | NA | 3.800 | 0.100 |
| LBXPFOA | 2005-2006 | LBDPFOAL | 2120 | 0 | 0 | NA | 3.700 | 0.070 |
| LBXPFOA | 2007-2008 | LBDPFOAL | 2100 | 0 | 0 | NA | 4.200 | 0.070 |
| LBXPFOA | 2009-2010 | LBDPFOAL | 2233 | 0 | 0 | NA | 2.900 | 0.070 |
| LBXPFOA | 2011-2012 | LBDPFOAL | 1904 | 0 | 0 | NA | 2.050 | 0.070 |
| LBXPFOA | 2013-2014 | LBDPFOAL | 0 | 0 | NA | NA | NA | NA |
| LBXPFOA | 2015-2016 | LBDPFOAL | 1993 | 0 | 0 | NA | 1.470 | 0.140 |
| LBXPFOA | 2017-2018 | LBDPFOAL | 1929 | 0 | 0 | NA | 1.370 | 0.140 |
| LBXPFNA | 2003-2004 | LBDPFNAL | 2094 | 0 | 0 | NA | 0.900 | 0.100 |
| LBXPFNA | 2005-2006 | LBDPFNAL | 2120 | 0 | 0 | NA | 1.000 | 0.070 |
| LBXPFNA | 2007-2008 | LBDPFNAL | 2100 | 0 | 0 | NA | 1.148 | 0.058 |

| exposure | cycle | lc_indicator | n | n_below | pct_below | median_below | median_above | min_positive |
|---|---|---|---|---|---|---|---|---|
| LBXPFNA | 2009-2010 | LBDPFNAL | 2233 | 0 | 0 | NA | 1.148 | 0.058 |
| LBXPFNA | 2011-2012 | LBDPFNAL | 1904 | 0 | 0 | NA | 0.890 | 0.060 |
| LBXPFNA | 2013-2014 | LBDPFNAL | 2168 | 0 | 0 | NA | 0.700 | 0.070 |
| LBXPFNA | 2015-2016 | LBDPFNAL | 1993 | 0 | 0 | NA | 0.600 | 0.070 |
| LBXPFNA | 2017-2018 | LBDPFNAL | 1929 | 0 | 0 | NA | 0.400 | 0.070 |
| LBXPFHS | 2003-2004 | LBDPFHSL | 2094 | 0 | 0 | NA | 1.900 | 0.200 |
| LBXPFHS | 2005-2006 | LBDPFHSL | 2120 | 0 | 0 | NA | 1.700 | 0.070 |
| LBXPFHS | 2007-2008 | LBDPFHSL | 2100 | 0 | 0 | NA | 2.000 | 0.070 |
| LBXPFHS | 2009-2010 | LBDPFHSL | 2233 | 0 | 0 | NA | 1.600 | 0.070 |
| LBXPFHS | 2011-2012 | LBDPFHSL | 1904 | 0 | 0 | NA | 1.190 | 0.070 |
| LBXPFHS | 2013-2014 | LBDPFHSL | 2168 | 0 | 0 | NA | 1.300 | 0.070 |
| LBXPFHS | 2015-2016 | LBDPFHSL | 1993 | 0 | 0 | NA | 1.200 | 0.070 |
| LBXPFHS | 2017-2018 | LBDPFHSL | 1929 | 0 | 0 | NA | 1.100 | 0.070 |

## Analytic Sample and Survey Design

```
analytic <- dat %>%
  filter(age >= config$min_age)

# PFAS workflow: harmonize rotating 2-year subsample weights into one unified PFAS weight (WTPFAS2YR).
analytic <- analytic %>%
  mutate(
    WTPFAS2YR = coalesce_num(cur_data(), c("WTSA2YR", "WTSB2YR", "WTSC2YR", "WTSAF2YR"))
  )

weight_candidates <- names(analytic)[stringr::str_detect(names(analytic), "^WT")]
kable(data.frame(weight_variable = weight_candidates), caption = "Weight audit: all WT* variables in ana
```

Table 17: Weight audit: all WT* variables in analytic dataset.

| weight_variable |
|---|
| WTINT2YR |
| WTMEC2YR |
| WTSA2YR |
| WTSAF2YR |

| weight_variable |
|---|
| WTSB2YR |
| WTSC2YR |
| WTPFAS2YR |

```
# Exposure-specific weight audit in the exposure-complete analytic sample (fixed primary PFAS set).
complete_expo <- analytic %>%
  mutate(.has_all_expo = if_all(all_of(primary_exposure_vars), ~ !is.na(.x)))
wt_cols <- names(analytic)[str_detect(names(analytic), "^WT")]
# PFAS auto-selection eligibility: exclude full-sample interview/MEC weights; keep PFAS-appropriate sub
if (tolower(config$exposure_family) == "pfas") {
  wt_cols <- setdiff(wt_cols, c("WTINT2YR", "WTMEC2YR"))
}
if (length(wt_cols) > 0) {
  wt_audit <- complete_expo %>%
    filter(.has_all_expo) %>%
    summarise(across(all_of(wt_cols), ~ round(mean(!is.na(.x)) * 100, 1))) %>%
    pivot_longer(everything(), names_to = "weight", values_to = "pct_nonmissing_complete_expo") %>%
    arrange(desc(pct_nonmissing_complete_expo))
  kable(wt_audit, caption = "Weight availability (%) among participants with complete exposure set. Use
}
```

Table 18: Weight availability (%) among participants with complete exposure set. Use the weight with highest coverage that matches your subsample.

| weight | pct_nonmissing_complete_expo |
|---|---|
| WTPFAS2YR | 100.0 |
| WTSAF2YR | 42.6 |
| WTSA2YR | 39.6 |
| WTSC2YR | 31.8 |
| WTSB2YR | 28.7 |

```
# Auto-weight: rank eligible candidate weights by coverage and apply the top PFAS-appropriate subsample
if (config$weight_type == "auto" && exists("wt_audit") && nrow(wt_audit) > 0) {
  wt_audit_auto <- wt_audit
  if (tolower(config$exposure_family) == "pfas") {
    wt_audit_auto <- wt_audit_auto %>% filter(!(weight %in% c("WTINT2YR", "WTMEC2YR")))
  }
  if (nrow(wt_audit_auto) == 0) {
    stop("Auto-weight selection failed for PFAS: no eligible WTS* subsample weights were available in t
  }
  top_row <- wt_audit_auto %>% slice(1)
  config$weight_type <- top_row$weight
  message(">>> Auto-weight resolved: Applying '", config$weight_type, "' (Coverage: ", top_row$pct_nonm
}

weight_info <- resolve_weight_var(analytic, config$weight_type, config$exposure_family)

analytic$weight_2yr <- suppressWarnings(as.numeric(as.character(analytic[[weight_info$var]])))
```

```r
analytic <- analytic %>%
  filter(!is.na(phenoage), !is.na(age), !is.na(ckd), !is.na(weight_2yr), !is.na(strata), !is.na(psu))

n_cycles <- length(unique(analytic$cycle))
# After resolving the PFAS-appropriate 2-year weight, pooled analyses divide by the number of contribut
n_cycles_exposure_available <- analytic %>%
  filter(if_any(all_of(primary_exposure_vars), ~ !is.na(.x))) %>%
  summarise(n = n_distinct(cycle)) %>%
  pull(n)
if (!is.finite(n_cycles_exposure_available) || n_cycles_exposure_available < 1) {
  n_cycles_exposure_available <- n_cycles
}
repool_weights <- function(df) {
  ncy <- dplyr::n_distinct(df$cycle[!is.na(df$weight_2yr)])
  if (!is.finite(ncy) || ncy < 1) stop("Invalid cycle count for weight pooling.")
  df$weight_combined <- df$weight_2yr / ncy
  df
}
analytic <- repool_weights(analytic)
message("Weight pooling: repool_weights() applied to analytic (n_cycles in sample = ", n_cycles_exposure

# Audit B: weight availability in the final analytic sample (after all exclusions).
if (length(wt_cols) > 0) {
  wt_audit_final <- analytic %>%
    summarise(across(any_of(wt_cols), ~ round(mean(!is.na(.x)) * 100, 1))) %>%
    pivot_longer(everything(), names_to = "weight", values_to = "pct_nonmissing_final_sample") %>%
    arrange(desc(pct_nonmissing_final_sample))
  kable(wt_audit_final, caption = "Weight availability (%) in final analytic sample (post-exclusion). T
}
```

Table 19: Weight availability (%) in final analytic sample (post-exclusion). This is the definitive audit.

| weight | pct_nonmissing_final_sample |
|---|---|
| WTPFAS2YR | 100.0 |
| WTSAF2YR | 74.1 |
| WTSC2YR | 22.1 |
| WTSB2YR | 19.5 |
| WTSA2YR | 9.0 |

```r
# PRIMARY: survey-weighted residualization of PhenoAge on age.
# This ensures PhenoAgeAccel is a population-representative construct.
analytic$phenoage_accel_unweighted <- NA_real_
fit_age_uw <- lm(phenoage ~ age, data = analytic)
analytic$phenoage_accel_unweighted <- residuals(fit_age_uw)
keep_w <- which(!is.na(analytic$phenoage) & !is.na(analytic$age) & !is.na(analytic$weight_combined))
if (length(keep_w) > 0) {
  svy_resid <- svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined, nest = TRUE,
                         data = analytic[keep_w, , drop = FALSE])
  fit_age_wt <- svyglm(phenoage ~ age, design = svy_resid, family = gaussian())
  pred_wt <- as.numeric(predict(fit_age_wt))
  analytic$phenoage_accel <- NA_real_
```

```r
  analytic$phenoage_accel[keep_w] <- analytic$phenoage[keep_w] - pred_wt
}
analytic <- analytic %>% filter(!is.na(phenoage_accel))
message(">>> PhenoAgeAccel: primary = survey-weighted residualization; unweighted version retained as s

ckd_marker_availability <- tibble::tibble(
  group = c("Both eGFR and ACR", "eGFR only", "ACR only", "Neither"),
  N = c(
    sum(!is.na(analytic$egfr) & !is.na(analytic$acr)),
    sum(!is.na(analytic$egfr) & is.na(analytic$acr)),
    sum(is.na(analytic$egfr) & !is.na(analytic$acr)),
    sum(is.na(analytic$egfr) & is.na(analytic$acr))
  )
)

if (!all(c(0, 1) %in% unique(analytic$ckd))) {
  stop("Both CKD groups are not represented after exclusions.")
}

svy <- svydesign(
  id = ~psu, strata = ~strata, weights = ~weight_combined, nest = TRUE, data = analytic
)

kable(data.frame(weight_selected = weight_info$var, note = weight_info$note), caption = "Selected NHANES
```

Table 20: Selected NHANES weight variable

| weight_selected | note |
|---|---|
| WTPFAS2YR | User-specified weight: WTPFAS2YR |

```r
kable(ckd_marker_availability, caption = "CKD marker availability in analytic sample")
```

Table 21: CKD marker availability in analytic sample

| group | N |
|---|---|
| Both eGFR and ACR | 15019 |
| eGFR only | 220 |
| ACR only | 0 |
| Neither | 0 |

# Mixture Modeling

```r
valid_num <- function(v, d) v %in% names(d) && length(unique(stats::na.omit(d[[v]]))) >= 2
valid_cat <- function(v, d) v %in% names(d) && length(unique(stats::na.omit(as.character(d[[v]])))) >= 

# Primary adjustment: lean set to avoid overadjustment/mediator bias (diabetes, HTN, CVD may be on path
# Aligned with EHP/Env Int mixture papers: sex, race, PIR, BMI, smoking, cycle only.
```

```r
clinical_covars <- c("diabetes", "hypertension", "cvd")  # used only in sensitivity "cardiometabolic"
# No age: outcome is phenoage_accel (age-residualized). Same 6 covariates for qgcomp and Model B.
covars <- c(
  if (valid_cat("sex", analytic)) "factor(sex)",
  if (valid_cat("race_eth", analytic)) "factor(race_eth)",
  if (valid_num("pir", analytic)) "pir",
  if (valid_num("smoker", analytic)) "smoker",
  if (valid_num("bmi", analytic)) "bmi",
  if (valid_cat("cycle", analytic)) "factor(cycle)"
)

# Sanity checks before modeling.
message("--- Pre-modeling sanity checks ---")
message("Analytic N: ", nrow(analytic))
message("PhenoAgeAccel: mean=", round(mean(analytic$phenoage_accel, na.rm = TRUE), 3),
        ", SD=", round(sd(analytic$phenoage_accel, na.rm = TRUE), 3))
message("CKD prevalence: ", round(mean(analytic$ckd == 1, na.rm = TRUE) * 100, 1), "%")
message("Weight selected: ", weight_info$var)

# qgcomp complete-case: use primary (fixed) exposure vars with sufficient variability and non-missing i
qg_min_frac <- 0.5
qg_exposure_vars_for_cc <- primary_exposure_vars[vapply(primary_exposure_vars, function(v) {
  x <- suppressWarnings(as.numeric(analytic[[v]]))
  finite <- is.finite(x)
  n_ok <- sum(finite)
  pct_ok <- n_ok / max(1, nrow(analytic))
  n_ok > 0 && dplyr::n_distinct(x[finite]) >= 2 && pct_ok >= qg_min_frac
}, logical(1))]
if (length(qg_exposure_vars_for_cc) < 2) {
  qg_exposure_vars_for_cc <- primary_exposure_vars[vapply(primary_exposure_vars, function(v) {
    x <- suppressWarnings(as.numeric(analytic[[v]]))
    x <- x[is.finite(x)]
    length(x) > 0 && dplyr::n_distinct(x) >= 2
  }, logical(1))]
  if (length(qg_exposure_vars_for_cc) < 2) qg_exposure_vars_for_cc <- primary_exposure_vars
  message("qgcomp: few primary exposures met variance + ", round(qg_min_frac * 100), "% non-missing; usi
}
message("qgcomp exposures for complete-case (primary fixed set, variance + >= ", round(qg_min_frac * 10
# Qgcomp uses same 6-variable adjustment as Model B (sex, race, PIR, BMI, smoking, cycle); no age (outc
rhs_qg_cc <- paste(c(qg_exposure_vars_for_cc, covars), collapse = " + ")
f_qg_cc <- as.formula(paste("phenoage_accel ~", rhs_qg_cc))
qg_cc <- complete.cases(model.frame(f_qg_cc, data = analytic, na.action = na.pass))
analytic_qg <- analytic[qg_cc, , drop = FALSE]
analytic_qg <- repool_weights(analytic_qg)
# Normalize precision weights for qgcomp so glm does not treat sum(weights) as N (avoids N = 300M, infi
analytic_qg$weight_normalized <- analytic_qg$weight_combined / mean(analytic_qg$weight_combined, na.rm =
# Ensure covariates are proper type for qgcomp (character -> wrong model matrix, psi ~ 0).
for (v in c("phenoage_accel", "age", "pir", "smoker", "bmi")) {
  if (v %in% names(analytic_qg)) analytic_qg[[v]] <- suppressWarnings(as.numeric(as.character(analytic_
}
analytic_qg$sex      <- factor(as.character(analytic_qg$sex))
analytic_qg$race_eth <- factor(as.character(analytic_qg$race_eth))
analytic_qg$cycle    <- factor(as.character(analytic_qg$cycle))
```

```r
for (v in qg_exposure_vars_for_cc) {
  if (v %in% names(analytic_qg)) analytic_qg[[v]] <- suppressWarnings(as.numeric(as.character(analytic_
}
message("qgcomp complete-case: same 6 covars as Model B; N = ", nrow(analytic_qg))

# Among those, keep only exposures that still have variance in the complete-case sample (in case one co
qg_exposure_vars <- qg_exposure_vars_for_cc[vapply(qg_exposure_vars_for_cc, function(v) {
  x <- suppressWarnings(as.numeric(analytic_qg[[v]]))
  x <- x[is.finite(x)]
  length(x) > 0 && dplyr::n_distinct(x) >= 2
}, logical(1))]
if (length(qg_exposure_vars) < 2) {
  warning("Too few finite/variable exposures for qgcomp after complete-case filtering; Table 2 will be
}
if (length(qg_exposure_vars) > 0) {
  analytic_qg <- analytic_qg %>%
    filter(if_all(all_of(c("phenoage_accel", qg_exposure_vars)), ~ is.finite(.x)))
}
q_qg <- as.integer(config$q)
if (!is.finite(q_qg) || q_qg < 2) stop("config$q must be an integer >= 2.")

# Qgcomp model: same 6-variable adjustment as Model B (no age).
covars_qg <- c(
  if (valid_cat("sex", analytic_qg)) "factor(sex)",
  if (valid_cat("race_eth", analytic_qg)) "factor(race_eth)",
  if (valid_num("pir", analytic_qg)) "pir",
  if (valid_num("smoker", analytic_qg)) "smoker",
  if (valid_num("bmi", analytic_qg)) "bmi",
  if (valid_cat("cycle", analytic_qg)) "factor(cycle)"
)
qg_covars_intended <- covars_qg
build_qg_formula <- function(exps, covs) {
  rhs <- paste(c(exps, covs), collapse = " + ")
  as.formula(paste("phenoage_accel ~", rhs))
}
f_qg <- build_qg_formula(qg_exposure_vars, covars_qg)

repeat {
  mf_qg <- tryCatch(model.frame(f_qg, data = analytic_qg, na.action = na.omit), error = function(e) NULL
  if (is.null(mf_qg) || nrow(mf_qg) == 0) break
  factor_terms_qg <- names(mf_qg)[vapply(mf_qg, is.factor, logical(1))]
  bad_factor_terms_qg <- factor_terms_qg[vapply(mf_qg[factor_terms_qg], function(x) nlevels(droplevels(
  removable_qg <- intersect(covars_qg, bad_factor_terms_qg)
  if (length(removable_qg) == 0) break
  message("qgcomp overall: dropping collapsed factor term(s): ", paste(removable_qg, collapse = ", "))
  covars_qg <- setdiff(covars_qg, removable_qg)
  f_qg <- build_qg_formula(qg_exposure_vars, covars_qg)
}
qg_covars_realized <- covars_qg
message("qgcomp exposures used: ", paste(qg_exposure_vars, collapse = ", "))
message("qgcomp finite analytic N: ", nrow(analytic_qg))

qgcomp_overall_error <- NULL
```

```r
fit_qg_overall <- tryCatch(
  qgcomp::qgcomp.noboot(
    f = f_qg, expnms = qg_exposure_vars, data = analytic_qg,
    family = gaussian(), q = q_qg, weights = analytic_qg$weight_normalized
  ),
  error = function(e) {
    qgcomp_overall_error <<- conditionMessage(e)
    NULL
  }
)
if (!is.null(fit_qg_overall)) {
  message("qgcomp N used: ", fit_qg_overall$fit$n[1])
} else {
  warning("qgcomp overall model failed: ", qgcomp_overall_error)
}

# Survey-weighted interaction model using quantized summed score.
# Quantile cutpoints are computed within each NHANES cycle using cycle-specific sampling weights.
qdat <- analytic
qdat$ckd <- as.integer(qdat$ckd)
# Strict sample = complete-case on qgcomp-compatible PFAS subset (same as analytic_qg). Label reflects
qdat_strict <- analytic_qg
qdat_strict$ckd <- as.integer(qdat_strict$ckd)

# Primary model: fixed PFAS set (primary_exposure_vars). Restrict to complete case on primary set.
qdat <- qdat %>% filter(if_all(all_of(primary_exposure_vars), ~ !is.na(.x)))
if (config$require_complete_exposure_set) {
  # Optional extra: could also require full adaptive set here; we use primary only for main.
}
message("Primary interaction: fixed PFAS set = ", paste(primary_exposure_vars, collapse = ", "))
message("N primary pre-quantile sample: ", nrow(qdat))
message("N strict sample (Strict complete-case, qgcomp-compatible PFAS subset): ", nrow(qdat_strict))

weighted_quantile <- function(x, w, probs) {
  keep <- is.finite(x) & is.finite(w) & w > 0
  x <- x[keep]; w <- w[keep]
  if (length(x) == 0) return(rep(NA_real_, length(probs)))
  ord <- order(x)
  x <- x[ord]; w <- w[ord]
  cw <- cumsum(w) / sum(w)
  sapply(probs, function(p) {
    idx <- which(cw >= p)[1]
    if (is.na(idx)) x[length(x)] else x[idx]
  })
}

# Primary mixture score: fixed set, cycle-specific quantiles to avoid temporal confounding (PFAS trends
for (v in primary_exposure_vars) {
  qdat[[paste0(v, "_q")]] <- NA_real_
  for (cy in unique(qdat$cycle)) {
    idx <- qdat$cycle == cy
    x_cy <- suppressWarnings(as.numeric(qdat[[v]][idx]))
    w_cy <- suppressWarnings(as.numeric(qdat$weight_combined[idx]))
```

```r
    probs <- weighted_quantile(x_cy, w_cy, probs = seq(0, 1, length.out = config$q + 1))
    br <- sort(unique(as.numeric(probs)))
    if (length(br) > 1) {
      br[1] <- -Inf
      br[length(br)] <- Inf
    }
    if (length(br) < 3 || any(diff(br) <= 0)) {
      r <- rank(qdat[[v]][idx], na.last = "keep", ties.method = "average")
      qdat[[paste0(v, "_q")]][idx] <- as.numeric(cut(r, breaks = config$q, labels = FALSE, include.lowe
    } else {
      qdat[[paste0(v, "_q")]][idx] <- as.numeric(cut(qdat[[v]][idx], breaks = br, include.lowest = TRUE
    }
  }
}
qvars <- paste0(primary_exposure_vars, "_q")
qdat$nonmissing_q_exposures <- rowSums(!is.na(qdat[, qvars, drop = FALSE]))
qdat$mixture_qscore <- rowMeans(qdat[, qvars, drop = FALSE], na.rm = TRUE)
# Primary: require all primary PFAS non-missing (fixed set).
required_primary <- length(primary_exposure_vars)
qdat$mixture_qscore <- ifelse(qdat$nonmissing_q_exposures < required_primary, NA_real_, qdat$mixture_qs
# By-cycle table for reporting (primary set is fixed; this documents it).
cycle_capacity <- tibble(
  cycle = unique(qdat$cycle),
  measured_exposures = length(primary_exposure_vars),
  PFAS_measured = paste(primary_exposure_vars, collapse = ", "),
  required_nonmissing_exposures = required_primary
)

# Build strict mixture score: same exposure set as qgcomp, cycle-specific quantiles.
for (v in qg_exposure_vars) {
  qdat_strict[[paste0(v, "_q")]] <- NA_real_
  for (cy in unique(qdat_strict$cycle)) {
    idx <- qdat_strict$cycle == cy
    x_cy <- suppressWarnings(as.numeric(qdat_strict[[v]][idx]))
    w_cy <- suppressWarnings(as.numeric(qdat_strict$weight_combined[idx]))
    probs_s <- weighted_quantile(x_cy, w_cy, probs = seq(0, 1, length.out = config$q + 1))
    br_s <- sort(unique(as.numeric(probs_s)))
    if (length(br_s) > 1) {
      br_s[1] <- -Inf
      br_s[length(br_s)] <- Inf
    }
    if (length(br_s) < 3 || any(diff(br_s) <= 0)) {
      r_s <- rank(qdat_strict[[v]][idx], na.last = "keep", ties.method = "average")
      qdat_strict[[paste0(v, "_q")]][idx] <- as.numeric(cut(r_s, breaks = config$q, labels = FALSE, incl
    } else {
      qdat_strict[[paste0(v, "_q")]][idx] <- as.numeric(cut(qdat_strict[[v]][idx], breaks = br_s, inclu
    }
  }
}
qvars_strict <- paste0(qg_exposure_vars, "_q")
qdat_strict$nonmissing_q_exposures <- rowSums(!is.na(qdat_strict[, qvars_strict, drop = FALSE]))
qdat_strict$mixture_qscore <- rowMeans(qdat_strict[, qvars_strict, drop = FALSE], na.rm = TRUE)
```

```r
# Hard type enforcement to prevent silent coercion issues in svyglm.
qdat <- qdat %>%
  mutate(
    phenoage_accel = suppressWarnings(as.numeric(phenoage_accel)),
    mixture_qscore = suppressWarnings(as.numeric(mixture_qscore)),
    weight_combined = suppressWarnings(as.numeric(weight_combined)),
    psu = suppressWarnings(as.numeric(psu)),
    strata = suppressWarnings(as.numeric(strata)),
    ckd = as.integer(ckd)
  )
qdat_strict <- qdat_strict %>%
  mutate(
    phenoage_accel = suppressWarnings(as.numeric(phenoage_accel)),
    mixture_qscore = suppressWarnings(as.numeric(mixture_qscore)),
    weight_combined = suppressWarnings(as.numeric(weight_combined)),
    psu = suppressWarnings(as.numeric(psu)),
    strata = suppressWarnings(as.numeric(strata)),
    ckd = as.integer(ckd)
  )

# Primary model: fixed PFAS set (same in every cycle).
cycle_capacity_with_min <- cycle_capacity
kable(
  cycle_capacity_with_min,
  caption = "Primary analysis: fixed PFAS set by cycle. Same chemicals and required non-missing in all c
)
```

Table 22: Primary analysis: fixed PFAS set by cycle. Same chemicals and required non-missing in all cycles.

| cycle | measured_exposures | PFAS_measured | required_nonmissing_exposures |
|---|---|---|---|
| 2005-2006 | 4 | LBXPFOS_log2, LBXPFOA_log2, LBXPFNA_log2, LBXPFHS_log2 | 4 |
| 2007-2008 | 4 | LBXPFOS_log2, LBXPFOA_log2, LBXPFNA_log2, LBXPFHS_log2 | 4 |
| 2009-2010 | 4 | LBXPFOS_log2, LBXPFOA_log2, LBXPFNA_log2, LBXPFHS_log2 | 4 |
| 2015-2016 | 4 | LBXPFOS_log2, LBXPFOA_log2, LBXPFNA_log2, LBXPFHS_log2 | 4 |
| 2017-2018 | 4 | LBXPFOS_log2, LBXPFOA_log2, LBXPFNA_log2, LBXPFHS_log2 | 4 |

```r
kable(
  tibble(
    statistic = c("N", "Mean", "SD", "Min", "P25", "Median", "P75", "Max"),
    value = c(
      sum(!is.na(qdat$mixture_qscore)),
      round(mean(qdat$mixture_qscore, na.rm = TRUE), 3),
      round(sd(qdat$mixture_qscore, na.rm = TRUE), 3),
      round(min(qdat$mixture_qscore, na.rm = TRUE), 3),
      round(quantile(qdat$mixture_qscore, 0.25, na.rm = TRUE), 3),
      round(median(qdat$mixture_qscore, na.rm = TRUE), 3),
```

```
      round(quantile(qdat$mixture_qscore, 0.75, na.rm = TRUE), 3),
      round(max(qdat$mixture_qscore, na.rm = TRUE), 3)
    )
  ),
  caption = paste0("Primary mixture quantile score (fixed set, N chemicals = ", length(primary_exposure
                   "). Score = mean quantile bin across ", paste(primary_exposure_vars, collapse = ", "
                   ". Complete case on primary set required.")
)
```

Table 23: Primary mixture quantile score (fixed set, N chemicals = 4). Score = mean quantile bin across LBXPFOS_log2, LBXPFOA_log2, LBXPFNA_log2, LBXPFHS_log2. Complete case on primary set required.

| statistic | value |
|---|---|
| N | 7633.000 |
| Mean | 2.419 |
| SD | 0.933 |
| Min | 1.000 |
| P25 | 1.500 |
| Median | 2.500 |
| P75 | 3.250 |
| Max | 4.000 |

```
stopifnot(all(c("psu", "strata", "weight_combined") %in% names(qdat)))
qdat <- qdat %>% filter(!is.na(psu), !is.na(strata), !is.na(weight_combined), !is.na(phenoage_accel), !
qdat_strict <- qdat_strict %>% filter(!is.na(psu), !is.na(strata), !is.na(weight_combined), !is.na(phen
qdat         <- repool_weights(qdat)
qdat_strict  <- repool_weights(qdat_strict)

# Ensure all model variables are proper type so svyglm does not zero out coefficients (character -> wro
for (v in exposure_vars) {
  if (v %in% names(qdat)) qdat[[v]] <- suppressWarnings(as.numeric(as.character(qdat[[v]])))
}
qdat$phenoage_accel <- suppressWarnings(as.numeric(as.character(qdat$phenoage_accel)))
qdat$age            <- suppressWarnings(as.numeric(as.character(qdat$age)))
qdat$pir            <- suppressWarnings(as.numeric(as.character(qdat$pir)))
qdat$smoker         <- suppressWarnings(as.numeric(as.character(qdat$smoker)))
qdat$bmi            <- suppressWarnings(as.numeric(as.character(qdat$bmi)))
qdat$sex            <- factor(as.character(qdat$sex))
qdat$race_eth       <- factor(as.character(qdat$race_eth))
qdat$cycle          <- factor(as.character(qdat$cycle))
qdat$ckd            <- as.integer(qdat$ckd)

for (v in exposure_vars) {
  if (v %in% names(qdat_strict)) qdat_strict[[v]] <- suppressWarnings(as.numeric(as.character(qdat_stri
}
qdat_strict$phenoage_accel <- suppressWarnings(as.numeric(as.character(qdat_strict$phenoage_accel)))
qdat_strict$age            <- suppressWarnings(as.numeric(as.character(qdat_strict$age)))
qdat_strict$pir            <- suppressWarnings(as.numeric(as.character(qdat_strict$pir)))
qdat_strict$smoker         <- suppressWarnings(as.numeric(as.character(qdat_strict$smoker)))
qdat_strict$bmi            <- suppressWarnings(as.numeric(as.character(qdat_strict$bmi)))
```

```r
qdat_strict$sex            <- factor(as.character(qdat_strict$sex))
qdat_strict$race_eth       <- factor(as.character(qdat_strict$race_eth))
qdat_strict$cycle          <- factor(as.character(qdat_strict$cycle))
qdat_strict$ckd            <- as.integer(qdat_strict$ckd)

message("---- Model B diagnostics ----")
message("N qdat: ", nrow(qdat))
print(table(qdat$ckd, useNA = "ifany"))
```

```
##
##    0    1
## 6315 1318
```

```r
message("N qdat_strict: ", nrow(qdat_strict))
print(table(qdat_strict$ckd, useNA = "ifany"))
```

```
##
##    0    1
## 2473  585
```

```r
sd_all <- sd(qdat$mixture_qscore, na.rm = TRUE)
sd_by <- tapply(qdat$mixture_qscore, qdat$ckd, sd, na.rm = TRUE)
message("SD(mixture_qscore) overall: ", sd_all)
print(sd_by)
```

```
##         0         1
## 0.9268457 0.9599365
```

```r
if (length(unique(stats::na.omit(qdat$ckd))) < 2) {
  stop("After exposure/design filtering, only one CKD group remains in qdat. ",
       "Interaction mixture_qscore*ckd cannot be estimated. ",
       "Set config$require_complete_exposure_set = FALSE or reduce exposure set/cycles.")
}
if (any(is.finite(sd_by) & sd_by < 1e-8)) {
  stop("mixture_qscore has near-zero variance within at least one CKD group. ",
       "Interaction cannot be estimated; check quantile binning/ties and exposure variability.")
}

miss_compare_ckd <- qdat %>%
  mutate(
    retained_primary = complete.cases(phenoage_accel, mixture_qscore, ckd, age, sex, race_eth, pir, smo
  ) %>%
  group_by(ckd, retained_primary) %>%
  summarise(
    N = n(),
    mean_age = round(mean(age, na.rm = TRUE), 1),
    female_pct = round(mean(sex == "Female", na.rm = TRUE) * 100, 1),
    .groups = "drop"
  ) %>%
  group_by(ckd) %>%
  mutate(pct_within_ckd = round(100 * N / sum(N), 1)) %>%
```

```
  ungroup() %>%
  mutate(
    ckd = ifelse(ckd == 1, "CKD", "No CKD"),
    retained_primary = ifelse(retained_primary, "Retained in Model B", "Dropped from Model B")
  )
kable(miss_compare_ckd, caption = "Primary interaction model retention by CKD status (retained vs dropp
```

Table 24: Primary interaction model retention by CKD status (retained vs dropped participants).

| ckd | retained_primary | N | mean_age | female_pct | pct_within_ckd |
|---|---|---|---|---|---|
| No CKD | Dropped from Model B | 3842 | 46.0 | 58.8 | 60.8 |
| No CKD | Retained in Model B | 2473 | 48.7 | 41.2 | 39.2 |
| CKD | Dropped from Model B | 733 | 62.6 | 61.7 | 55.6 |
| CKD | Retained in Model B | 585 | 64.6 | 40.5 | 44.4 |

```
# Use explicit data-frame copy so survey design uses correct types (avoids ~0 coefficients from referen
qdat_svy <- as.data.frame(qdat)
qdat_svy$phenoage_accel <- as.numeric(qdat_svy$phenoage_accel)
qdat_svy$mixture_qscore <- as.numeric(qdat_svy$mixture_qscore)
qdat_svy$ckd            <- as.integer(qdat_svy$ckd)
qdat_svy$age            <- as.numeric(qdat_svy$age)
qdat_svy$pir            <- as.numeric(qdat_svy$pir)
qdat_svy$smoker         <- as.numeric(qdat_svy$smoker)
qdat_svy$bmi            <- as.numeric(qdat_svy$bmi)
qdat_svy$sex            <- factor(qdat_svy$sex)
qdat_svy$race_eth       <- factor(qdat_svy$race_eth)
qdat_svy$cycle          <- factor(qdat_svy$cycle)
qdat_svy$weight_combined <- as.numeric(qdat_svy$weight_combined)
qdat_svy$psu            <- as.numeric(qdat_svy$psu)
qdat_svy$strata         <- as.numeric(qdat_svy$strata)
# Drop unused factor levels to avoid singularity with full survey design.
qdat_svy$sex      <- droplevels(factor(qdat_svy$sex))
qdat_svy$race_eth <- droplevels(factor(qdat_svy$race_eth))
qdat_svy$cycle    <- droplevels(factor(qdat_svy$cycle))

# Full complex survey design (PSU/strata) for valid inference; droplevels above avoids empty cells.
svy_q <- tryCatch(
  svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined, nest = TRUE, data = qdat_svy),
  error = function(e) {
    message("Full design failed (", conditionMessage(e), "); falling back to weights-only design.")
    svydesign(id = ~1, strata = NULL, weights = ~weight_combined, data = qdat_svy)
  }
)
svy_q_strict <- if (nrow(qdat_strict) > 0) {
  qdat_strict_svy <- as.data.frame(qdat_strict)
  for (v in c("phenoage_accel", "mixture_qscore", "ckd", "age", "pir", "smoker", "bmi", "weight_combine
    if (v %in% names(qdat_strict_svy)) qdat_strict_svy[[v]] <- suppressWarnings(as.numeric(as.character
  }
  qdat_strict_svy$ckd <- as.integer(qdat_strict_svy$ckd)
  qdat_strict_svy$sex <- droplevels(factor(qdat_strict_svy$sex))
  qdat_strict_svy$race_eth <- droplevels(factor(qdat_strict_svy$race_eth))
```

```r
  qdat_strict_svy$cycle <- droplevels(factor(qdat_strict_svy$cycle))
  tryCatch(
    svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined, nest = TRUE, data = qdat_strict_s
    error = function(e) svydesign(id = ~1, strata = NULL, weights = ~weight_combined, data = qdat_stric
  )
} else {
  NULL
}

# Model B: outcome is phenoage_accel (already age-residualized); do not adjust for age again (no double
covars_int <- c(
  if (valid_cat("sex", qdat)) "factor(sex)",
  if (valid_cat("race_eth", qdat)) "factor(race_eth)",
  if (valid_num("pir", qdat)) "pir",
  if (valid_num("smoker", qdat)) "smoker",
  if (valid_num("bmi", qdat)) "bmi",
  if (valid_cat("cycle", qdat)) "factor(cycle)"
)
intended_covars_int <- covars_int
build_int_formula <- function(covars_vec) {
  rhs <- if (length(covars_vec) > 0) paste("+", paste(covars_vec, collapse = " + ")) else ""
  as.formula(paste("phenoage_accel ~ mixture_qscore * ckd", rhs))
}
f_int <- build_int_formula(covars_int)

repeat {
  mf_int <- tryCatch(model.frame(f_int, data = qdat, na.action = na.omit), error = function(e) NULL)
  if (is.null(mf_int) || nrow(mf_int) == 0) {
    stop("Model B preprocessing failed: no complete-case rows available for interaction formula.")
  }
  factor_terms <- names(mf_int)[vapply(mf_int, is.factor, logical(1))]
  bad_factor_terms <- factor_terms[vapply(mf_int[factor_terms], function(x) nlevels(droplevels(x)) < 2,
  removable_terms <- intersect(covars_int, bad_factor_terms)
  if (length(removable_terms) == 0) break
  message("Model B: dropping collapsed factor term(s) after complete-case filtering: ",
          paste(removable_terms, collapse = ", "))
  covars_int <- setdiff(covars_int, removable_terms)
  f_int <- build_int_formula(covars_int)
}
n_modelb <- nrow(mf_int)
realized_covars_int <- covars_int
covar_report <- tibble(
  intended_term = intended_covars_int,
  realized_in_model_b = ifelse(intended_covars_int %in% realized_covars_int, "Yes", "No")
)
kable(covar_report, caption = "Model B covariate transparency: intended vs realized adjustment terms.")
```

Table 25: Model B covariate transparency: intended vs realized adjustment terms.

| intended_term | realized_in_model_b |
|---|---|
| factor(sex) | Yes |

| intended_term | realized_in_model_b |
|---|---|
| factor(race_eth) | Yes |
| pir | Yes |
| smoker | Yes |
| bmi | Yes |
| factor(cycle) | Yes |

```
fit_int_error <- NULL
fit_int <- tryCatch(
  svyglm(f_int, design = svy_q, family = gaussian()),
  error = function(e) {
    fit_int_error <<- conditionMessage(e)
    NULL
  }
)
if (is.null(fit_int)) {
  stop("Model B (svyglm interaction) failed: ", fit_int_error)
} else {
  message("Model B fit OK. Coefficients:")
  message("Model B complete-case N: ", n_modelb)
  print(coef(fit_int))
}
```

```
##                        (Intercept)                     mixture_qscore
##                        -10.2304717                        -0.7731055
##                                ckd                     factor(sex)Male
##                          9.7160990                          2.1248385
## factor(race_eth)Non-Hispanic Asian factor(race_eth)Non-Hispanic Black
##                          0.2466249                          2.4308155
## factor(race_eth)Non-Hispanic White        factor(race_eth)Other/Multi
##                          0.2626766                         -0.5525380
##                                pir                             smoker
##                         -0.5863572                          2.3821428
##                                bmi             factor(cycle)2007-2008
##                          0.3269308                         -0.9323133
##             factor(cycle)2009-2010             factor(cycle)2015-2016
##                         -0.6667737                          4.2346060
##             factor(cycle)2017-2018                 mixture_qscore:ckd
##                          5.3702929                         -1.4514657
```

```
# Sanity check: weighted lm and svyglm (weights-only design) should match.
fit_int_lm <- lm(f_int, data = qdat_svy, weights = weight_combined)
message("Weighted lm mixture_qscore coef: ", round(coef(fit_int_lm)["mixture_qscore"], 6), "; svyglm: "

aqg0 <- analytic_qg %>% filter(ckd == 0)
aqg1 <- analytic_qg %>% filter(ckd == 1)
if (nrow(aqg0) > 0) aqg0$weight_normalized <- aqg0$weight_combined / mean(aqg0$weight_combined, na.rm =
if (nrow(aqg1) > 0) aqg1$weight_normalized <- aqg1$weight_combined / mean(aqg1$weight_combined, na.rm =
fit_qg_ckd0 <- tryCatch(
  qgcomp::qgcomp.noboot(
    f_qg, expnms = qg_exposure_vars,
    data = aqg0,
```

```
    family = gaussian(), q = q_qg,
    weights = aqg0$weight_normalized
  ),
  error = function(e) NULL
)
fit_qg_ckd1 <- tryCatch(
  qgcomp::qgcomp.noboot(
    f_qg, expnms = qg_exposure_vars,
    data = aqg1,
    family = gaussian(), q = q_qg,
    weights = aqg1$weight_normalized
  ),
  error = function(e) NULL
)
```

## Sample Flow (PRISMA-style)

```
n0 <- nrow(nhanes_raw)
n1 <- sum(dat$age >= config$min_age, na.rm = TRUE)
n2 <- sum(dat$age >= config$min_age & !is.na(dat$phenoage) & !is.na(dat$ckd), na.rm = TRUE)
n3 <- nrow(analytic)
n4 <- if (exists("qdat")) nrow(qdat) else NA_integer_
n5 <- if (exists("analytic_qg")) nrow(analytic_qg) else NA_integer_
n6 <- if (exists("qdat_strict")) nrow(qdat_strict) else NA_integer_
n7 <- if (exists("n_modelb")) n_modelb else NA_integer_

flow <- tibble(
  step = c(
    "Downloaded NHANES participants (selected cycles)",
    paste0("Age >= ", config$min_age),
    "Non-missing PhenoAge and CKD",
    "Non-missing selected weight, PSU, and strata (analytic sample)",
    "Primary interaction model sample (fixed 4-PFAS complete-case score + design vars)",
    "Complete-case qgcomp sample",
    "Strict complete-case (qgcomp-compatible PFAS subset)",
    "Model B complete-case estimation sample"
  ),
  included = c(n0, n1, n2, n3, n4, n5, n6, n7)
) %>%
  mutate(
    excluded_at_step = pmax(lag(included, default = first(included)) - included, 0),
    excluded_at_step = if_else(row_number() == 1, NA_integer_, as.integer(excluded_at_step)),
    excluded_label = ifelse(is.na(excluded_at_step), "", paste0("Excluded: ", excluded_at_step))
  )

kable(flow, caption = "Sample flow counts (PRISMA-style): inclusion and exclusion at each analytic step
```

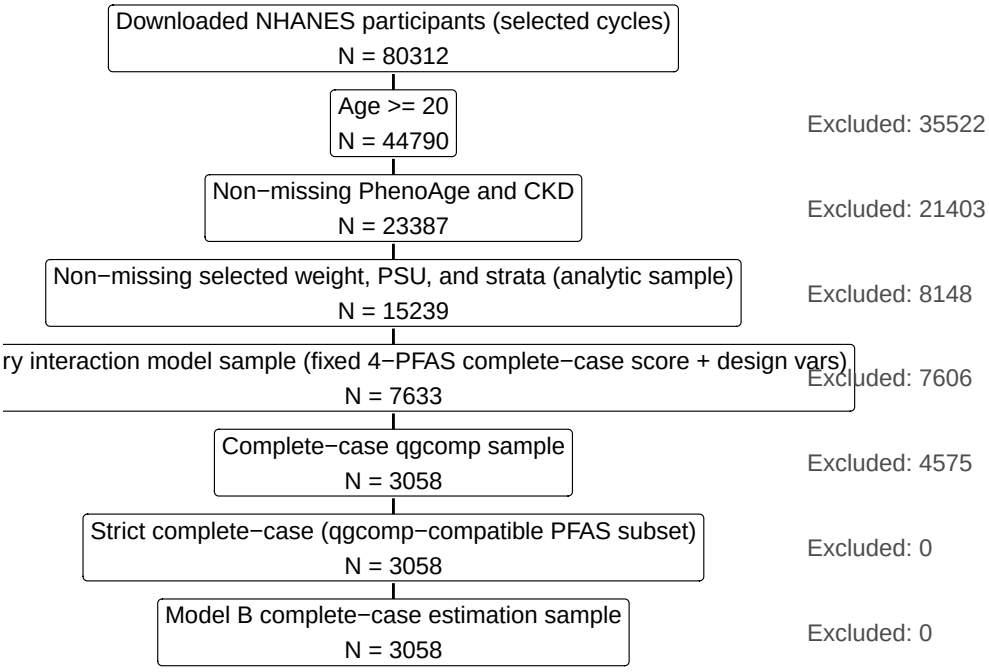
Table 26: Sample flow counts (PRISMA-style): inclusion and exclusion at each analytic step

| step | included | excluded_at_step | excluded_label |
|---|---|---|---|
| Downloaded NHANES participants (selected cycles) | 80312 | NA | |
| Age >= 20 | 44790 | 35522 | Excluded: 35522 |
| Non-missing PhenoAge and CKD | 23387 | 21403 | Excluded: 21403 |
| Non-missing selected weight, PSU, and strata (analytic sample) | 15239 | 8148 | Excluded: 8148 |
| Primary interaction model sample (fixed 4-PFAS complete-case score + design vars) | 7633 | 7606 | Excluded: 7606 |
| Complete-case qgcomp sample | 3058 | 4575 | Excluded: 4575 |
| Strict complete-case (qgcomp-compatible PFAS subset) | 3058 | 0 | Excluded: 0 |
| Model B complete-case estimation sample | 3058 | 0 | Excluded: 0 |

```r
flow_plot <- flow %>%
  mutate(
    y = rev(seq_len(n())),
    box_label = paste0(step, "\nN = ", included)
  )

ggplot(flow_plot, aes(x = 1, y = y)) +
  geom_segment(
    data = flow_plot %>% filter(y > min(y)),
    aes(x = 1, xend = 1, y = y - 0.42, yend = y - 0.95),
    arrow = arrow(length = unit(0.12, "inches")),
    linewidth = 0.5
  ) +
  geom_label(aes(label = box_label), size = 3.2, label.size = 0.2, fill = "white") +
  geom_text(aes(x = 1.62, label = excluded_label), hjust = 0, size = 3.1, color = "gray30") +
  xlim(0.5, 2.2) +
  ylim(0.5, max(flow_plot$y) + 0.8) +
  labs(
    title = "Participant flow from downloaded sample to analytic samples",
    subtitle = "PRISMA-style flow for NHANES analytic cohort development",
    x = NULL, y = NULL
  ) +
  theme_void(base_size = 11)
```

Participant flow from downloaded sample to analytic samples

PRISMA−style flow for NHANES analytic cohort development




# Tables

```
fmt <- function(x, digits = 3) {
  ifelse(
    is.na(x),
    "NA",
    ifelse(
      x != 0 & abs(x) < 10^-digits,
      format(x, scientific = TRUE, digits = 2),
      format(round(x, digits), nsmall = digits)
    )
  )
}
fmt_p <- function(x) ifelse(is.na(x), "NA", ifelse(x < 0.001, "<0.001", format(round(x, 3), nsmall = 3)

# Table 1: characteristics by CKD -- continuous: mean (SD); categorical: n (weighted %)
t1_mean_sd <- function(formula, design) {
  est <- svymean(formula, design, na.rm = TRUE)
  v_obj <- svyvar(formula, design, na.rm = TRUE)
  sd_val <- sqrt(as.numeric(coef(v_obj)[1]))
  paste0(format(round(coef(est)[1], 2), nsmall = 2), " (", format(round(sd_val, 2), nsmall = 2), ")")
}
t1_mean_obj <- function(formula, design) {
```

```r
  m <- svymean(formula, design, na.rm = TRUE)
  v <- svyvar(formula, design, na.rm = TRUE)
  list(est = unname(coef(m)[1]), se = unname(SE(m)[1]), var = unname(coef(v)[1]))
}
t1_pct_se <- function(formula, design) {
  est <- svymean(formula, design, na.rm = TRUE)
  cf <- coef(est)
  se <- SE(est)
  idx <- 1
  if (length(cf) > 1) {
    nms <- names(cf)
    idx_true <- which(str_detect(nms, "TRUE$"))
    if (length(idx_true) == 0) idx_true <- which(str_detect(nms, "1$"))
    if (length(idx_true) > 0) idx <- idx_true[1]
  }
  paste0(
    format(round(unname(cf[idx]) * 100, 1), nsmall = 1),
    " (",
    format(round(unname(se[idx]) * 100, 1), nsmall = 1),
    ")"
  )
}
t1_n_wpct <- function(cond_vec, formula, design) {
  n_unwt <- sum(cond_vec, na.rm = TRUE)
  est <- svymean(formula, design, na.rm = TRUE)
  cf <- coef(est)
  idx <- 1
  if (length(cf) > 1) {
    nms <- names(cf)
    idx_true <- which(str_detect(nms, "TRUE$"))
    if (length(idx_true) == 0) idx_true <- which(str_detect(nms, "1$"))
    if (length(idx_true) > 0) idx <- idx_true[1]
  }
  paste0(
    format(n_unwt, big.mark = ","),
    " (",
    format(round(unname(cf[idx]) * 100, 1), nsmall = 1),
    ")"
  )
}
t1_pct_obj <- function(formula, design) {
  est <- svymean(formula, design, na.rm = TRUE)
  cf <- coef(est)
  se <- SE(est)
  idx <- 1
  if (length(cf) > 1) {
    nms <- names(cf)
    idx_true <- which(str_detect(nms, "TRUE$"))
    if (length(idx_true) == 0) idx_true <- which(str_detect(nms, "1$"))
    if (length(idx_true) > 0) idx <- idx_true[1]
  }
  list(est = unname(cf[idx]), se = unname(se[idx]))
}
```

```r
t1_factor_level_pct_se <- function(var, level, design) {
  est <- svymean(as.formula(paste0("~factor(", var, ")")), design, na.rm = TRUE)
  key <- paste0("factor(", var, ")", level)
  if (!(key %in% names(coef(est)))) return("NA")
  paste0(
    format(round(unname(coef(est)[key]) * 100, 1), nsmall = 1),
    " (",
    format(round(unname(SE(est)[key]) * 100, 1), nsmall = 1),
    ")"
  )
}
t1_factor_level_n_wpct <- function(var, level, design, data) {
  n_unwt <- sum(data[[var]] == level, na.rm = TRUE)
  est <- svymean(as.formula(paste0("~factor(", var, ")")), design, na.rm = TRUE)
  key <- paste0("factor(", var, ")", level)
  if (!(key %in% names(coef(est)))) {
    return(paste0(format(n_unwt, big.mark = ","), " (NA)"))
  }
  paste0(
    format(n_unwt, big.mark = ","),
    " (",
    format(round(unname(coef(est)[key]) * 100, 1), nsmall = 1),
    ")"
  )
}
t1_factor_level_obj <- function(var, level, design) {
  est <- svymean(as.formula(paste0("~factor(", var, ")")), design, na.rm = TRUE)
  key <- paste0("factor(", var, ")", level)
  if (!(key %in% names(coef(est)))) return(list(est = NA_real_, se = NA_real_))
  list(est = unname(coef(est)[key]), se = unname(SE(est)[key]))
}
smd_cont <- function(o0, o1) {
  if (any(!is.finite(c(o0$est, o1$est, o0$var, o1$var)))) return(NA_real_)
  denom <- sqrt((o0$var + o1$var) / 2)
  if (!is.finite(denom) || denom <= 0) return(NA_real_)
  abs(o1$est - o0$est) / denom
}
smd_prop <- function(o0, o1) {
  if (any(!is.finite(c(o0$est, o1$est)))) return(NA_real_)
  denom <- sqrt((o0$est * (1 - o0$est) + o1$est * (1 - o1$est)) / 2)
  if (!is.finite(denom) || denom <= 0) return(NA_real_)
  abs(o1$est - o0$est) / denom
}
fmt_smd <- function(x) ifelse(is.na(x), "", format(round(x, 3), nsmall = 3))

# Build Part A (Included = qgcomp complete-case, n = 3,058) and Part B (Excluded, n = 12,181)
# from the parent analytic sample (n = 15,239). Match by SEQN to the final analytic_qg used in modeling
if ("SEQN" %in% names(analytic) && exists("analytic_qg") && "SEQN" %in% names(analytic_qg)) {
  inc_idx <- analytic$SEQN %in% analytic_qg$SEQN
} else {
  inc_idx <- qg_cc
}
analytic_inc <- analytic[inc_idx, , drop = FALSE]
```

```r
analytic_exc <- analytic[!inc_idx, , drop = FALSE]

svy_inc <- svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined,
                     nest = TRUE, data = analytic_inc)
svy_exc <- svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined,
                     nest = TRUE, data = analytic_exc)

# Block builder: returns a tibble with columns characteristic, No CKD, CKD, SMD.
# Continuous: mean (SD).  Categorical: n (weighted %).  No SE displayed anywhere.
build_t1_block <- function(design, data) {
  d0 <- subset(design, ckd == 0)
  d1 <- subset(design, ckd == 1)
  data0 <- data[!is.na(data$ckd) & data$ckd == 0, , drop = FALSE]
  data1 <- data[!is.na(data$ckd) & data$ckd == 1, , drop = FALSE]

  age0 <- t1_mean_obj(~age, d0); age1 <- t1_mean_obj(~age, d1)
  pir0 <- t1_mean_obj(~pir, d0); pir1 <- t1_mean_obj(~pir, d1)
  bmi0 <- t1_mean_obj(~bmi, d0); bmi1 <- t1_mean_obj(~bmi, d1)

  fem0 <- t1_pct_obj(~I(sex == "Female"), d0); fem1 <- t1_pct_obj(~I(sex == "Female"), d1)
  his0 <- t1_factor_level_obj("race_eth", "Hispanic", d0); his1 <- t1_factor_level_obj("race_eth", "His
  whi0 <- t1_factor_level_obj("race_eth", "Non-Hispanic White", d0); whi1 <- t1_factor_level_obj("race_
  blk0 <- t1_factor_level_obj("race_eth", "Non-Hispanic Black", d0); blk1 <- t1_factor_level_obj("race_
  asn0 <- t1_factor_level_obj("race_eth", "Non-Hispanic Asian", d0); asn1 <- t1_factor_level_obj("race_
  oth0 <- t1_factor_level_obj("race_eth", "Other/Multi", d0); oth1 <- t1_factor_level_obj("race_eth", "
  edu_lths0 <- t1_factor_level_obj("education", "<HS", d0); edu_lths1 <- t1_factor_level_obj("education
  edu_hs0   <- t1_factor_level_obj("education", "HS/GED", d0); edu_hs1   <- t1_factor_level_obj("educat
  edu_gt0   <- t1_factor_level_obj("education", ">HS", d0); edu_gt1   <- t1_factor_level_obj("education
  smk0 <- t1_pct_obj(~I(smoker == 1), d0); smk1 <- t1_pct_obj(~I(smoker == 1), d1)
  alc0 <- t1_pct_obj(~I(alcohol == 1), d0); alc1 <- t1_pct_obj(~I(alcohol == 1), d1)
  pa0  <- t1_pct_obj(~I(phys_active == 1), d0); pa1  <- t1_pct_obj(~I(phys_active == 1), d1)
  dm0  <- t1_pct_obj(~I(diabetes == 1), d0); dm1  <- t1_pct_obj(~I(diabetes == 1), d1)
  htn0 <- t1_pct_obj(~I(hypertension == 1), d0); htn1 <- t1_pct_obj(~I(hypertension == 1), d1)
  cvd0 <- t1_pct_obj(~I(cvd == 1), d0); cvd1 <- t1_pct_obj(~I(cvd == 1), d1)

  tibble(
    characteristic = c(
      "N (unweighted)", "Age, mean (SD)", "Female, n (weighted %)",
      "Hispanic, n (weighted %)", "Non-Hispanic White, n (weighted %)", "Non-Hispanic Black, n (weighte
      "Non-Hispanic Asian, n (weighted %)", "Other/Multi, n (weighted %)",
      "Education <HS, n (weighted %)", "Education HS/GED, n (weighted %)", "Education >HS, n (weighted 
      "PIR, mean (SD)", "BMI, mean (SD)",
      "Current smoker, n (weighted %)", "Alcohol (yes), n (weighted %)", "Physically active, n (weighte
      "Diabetes, n (weighted %)", "Hypertension, n (weighted %)", "CVD history, n (weighted %)"
    ),
    `No CKD` = c(
      format(nrow(data0), big.mark = ","),
      t1_mean_sd(~age, d0),
      t1_n_wpct(data0$sex == "Female", ~I(sex == "Female"), d0),
      t1_factor_level_n_wpct("race_eth", "Hispanic", d0, data0),
      t1_factor_level_n_wpct("race_eth", "Non-Hispanic White", d0, data0),
      t1_factor_level_n_wpct("race_eth", "Non-Hispanic Black", d0, data0),
      t1_factor_level_n_wpct("race_eth", "Non-Hispanic Asian", d0, data0),
```

```
    t1_factor_level_n_wpct("race_eth", "Other/Multi", d0, data0),
    t1_factor_level_n_wpct("education", "<HS", d0, data0),
    t1_factor_level_n_wpct("education", "HS/GED", d0, data0),
    t1_factor_level_n_wpct("education", ">HS", d0, data0),
    t1_mean_sd(~pir, d0),
    t1_mean_sd(~bmi, d0),
    t1_n_wpct(data0$smoker == 1, ~I(smoker == 1), d0),
    t1_n_wpct(data0$alcohol == 1, ~I(alcohol == 1), d0),
    t1_n_wpct(data0$phys_active == 1, ~I(phys_active == 1), d0),
    t1_n_wpct(data0$diabetes == 1, ~I(diabetes == 1), d0),
    t1_n_wpct(data0$hypertension == 1, ~I(hypertension == 1), d0),
    t1_n_wpct(data0$cvd == 1, ~I(cvd == 1), d0)
  ),
  CKD = c(
    format(nrow(data1), big.mark = ","),
    t1_mean_sd(~age, d1),
    t1_n_wpct(data1$sex == "Female", ~I(sex == "Female"), d1),
    t1_factor_level_n_wpct("race_eth", "Hispanic", d1, data1),
    t1_factor_level_n_wpct("race_eth", "Non-Hispanic White", d1, data1),
    t1_factor_level_n_wpct("race_eth", "Non-Hispanic Black", d1, data1),
    t1_factor_level_n_wpct("race_eth", "Non-Hispanic Asian", d1, data1),
    t1_factor_level_n_wpct("race_eth", "Other/Multi", d1, data1),
    t1_factor_level_n_wpct("education", "<HS", d1, data1),
    t1_factor_level_n_wpct("education", "HS/GED", d1, data1),
    t1_factor_level_n_wpct("education", ">HS", d1, data1),
    t1_mean_sd(~pir, d1),
    t1_mean_sd(~bmi, d1),
    t1_n_wpct(data1$smoker == 1, ~I(smoker == 1), d1),
    t1_n_wpct(data1$alcohol == 1, ~I(alcohol == 1), d1),
    t1_n_wpct(data1$phys_active == 1, ~I(phys_active == 1), d1),
    t1_n_wpct(data1$diabetes == 1, ~I(diabetes == 1), d1),
    t1_n_wpct(data1$hypertension == 1, ~I(hypertension == 1), d1),
    t1_n_wpct(data1$cvd == 1, ~I(cvd == 1), d1)
  ),
  SMD = c(
    "",
    fmt_smd(smd_cont(age0, age1)),
    fmt_smd(smd_prop(fem0, fem1)),
    fmt_smd(smd_prop(his0, his1)),
    fmt_smd(smd_prop(whi0, whi1)),
    fmt_smd(smd_prop(blk0, blk1)),
    fmt_smd(smd_prop(asn0, asn1)),
    fmt_smd(smd_prop(oth0, oth1)),
    fmt_smd(smd_prop(edu_lths0, edu_lths1)),
    fmt_smd(smd_prop(edu_hs0, edu_hs1)),
    fmt_smd(smd_prop(edu_gt0, edu_gt1)),
    fmt_smd(smd_cont(pir0, pir1)),
    fmt_smd(smd_cont(bmi0, bmi1)),
    fmt_smd(smd_prop(smk0, smk1)),
    fmt_smd(smd_prop(alc0, alc1)),
    fmt_smd(smd_prop(pa0, pa1)),
    fmt_smd(smd_prop(dm0, dm1)),
    fmt_smd(smd_prop(htn0, htn1)),
```

```r
      fmt_smd(smd_prop(cvd0, cvd1))
    )
  )
}

block_a <- build_t1_block(svy_inc, analytic_inc)
block_b <- build_t1_block(svy_exc, analytic_exc)

n_inc <- nrow(analytic_inc)
n_exc <- nrow(analytic_exc)
n_inc_0 <- sum(analytic_inc$ckd == 0, na.rm = TRUE)
n_inc_1 <- sum(analytic_inc$ckd == 1, na.rm = TRUE)
n_exc_0 <- sum(analytic_exc$ckd == 0, na.rm = TRUE)
n_exc_1 <- sum(analytic_exc$ckd == 1, na.rm = TRUE)

header_a <- tibble(
  characteristic = paste0(
    "PART A -- Analytic sample, included (N = ", format(n_inc, big.mark = ","),
    "; No CKD = ", format(n_inc_0, big.mark = ","),
    ", CKD = ", format(n_inc_1, big.mark = ","), ")"
  ),
  `No CKD` = "", CKD = "", SMD = ""
)
header_b <- tibble(
  characteristic = paste0(
    "PART B -- Excluded participants (N = ", format(n_exc, big.mark = ","),
    "; No CKD = ", format(n_exc_0, big.mark = ","),
    ", CKD = ", format(n_exc_1, big.mark = ","), ")"
  ),
  `No CKD` = "", CKD = "", SMD = ""
)

table1 <- bind_rows(header_a, block_a, header_b, block_b)

kable(
  table1,
  caption = "Table 1. Sample characteristics by CKD status, stratified by inclusion in the complete-case
)
```

Table 27: Table 1. Sample characteristics by CKD status, stratified by inclusion in the complete-case modeling sample. Continuous: mean (SD); categorical: n (weighted %). SMD is the absolute standardized mean difference between CKD vs No CKD within each part.

| characteristic | No CKD | CKD | SMD |
|---|---|---|---|
| PART A – Analytic sample, included (N = 3,058; No CKD = 2,473, CKD = 585) | | | |
| N (unweighted) | 2,473 | 585 | |
| Age, mean (SD) | 46.21 (15.78) | 62.56 (15.56) | 1.043 |
| Female, n (weighted %) | 1,020 (42.6) | 237 (46.3) | 0.075 |

| characteristic | No CKD | CKD | SMD |
|---|---|---|---|
| Hispanic, n (weighted %) | 542 (10.7) | 112 (10.5) | 0.006 |
| Non-Hispanic White, n (weighted %) | 1,292 (74.0) | 297 (68.7) | 0.119 |
| Non-Hispanic Black, n (weighted %) | 459 (8.8) | 137 (13.0) | 0.136 |
| Non-Hispanic Asian, n (weighted %) | 61 (1.1) | 14 (1.4) | 0.032 |
| Other/Multi, n (weighted %) | 119 (5.4) | 25 (6.4) | 0.042 |
| Education <HS, n (weighted %) | 244 (5.6) | 92 (13.5) | 0.270 |
| Education HS/GED, n (weighted %) | 648 (33.7) | 133 (31.7) | 0.042 |
| Education >HS, n (weighted %) | 1,139 (60.6) | 253 (54.7) | 0.120 |
| PIR, mean (SD) | 2.89 (1.63) | 2.50 (1.58) | 0.247 |
| BMI, mean (SD) | 28.54 (6.69) | 30.45 (7.46) | 0.269 |
| Current smoker, n (weighted %) | 1,212 (48.5) | 178 (29.3) | 0.402 |
| Alcohol (yes), n (weighted %) | 212 (25.9) | 53 (27.7) | 0.041 |
| Physically active, n (weighted %) | 860 (50.6) | 157 (37.0) | 0.278 |
| Diabetes, n (weighted %) | 229 (7.3) | 208 (31.0) | 0.631 |
| Hypertension, n (weighted %) | 784 (28.2) | 393 (64.5) | 0.782 |
| CVD history, n (weighted %) | 244 (8.0) | 192 (28.8) | 0.556 |
| PART B – Excluded participants (N = 12,181; No CKD = 10,090, CKD = 2,091) | | | |
| N (unweighted) | 10,090 | 2,091 | |
| Age, mean (SD) | 44.91 (15.85) | 60.11 (17.64) | 0.906 |
| Female, n (weighted %) | 5,493 (53.8) | 1,160 (61.1) | 0.148 |
| Hispanic, n (weighted %) | 3,012 (15.5) | 508 (13.6) | 0.052 |
| Non-Hispanic White, n (weighted %) | 4,045 (65.3) | 897 (64.8) | 0.009 |
| Non-Hispanic Black, n (weighted %) | 1,950 (10.9) | 525 (14.8) | 0.117 |
| Non-Hispanic Asian, n (weighted %) | 611 (2.9) | 79 (2.3) | 0.041 |
| Other/Multi, n (weighted %) | 472 (5.5) | 82 (4.5) | 0.044 |
| Education <HS, n (weighted %) | 1,094 (5.8) | 362 (11.7) | 0.211 |
| Education HS/GED, n (weighted %) | 2,209 (23.9) | 543 (33.0) | 0.202 |
| Education >HS, n (weighted %) | 5,439 (70.3) | 836 (55.2) | 0.314 |
| PIR, mean (SD) | 3.17 (1.62) | 2.67 (1.56) | 0.317 |
| BMI, mean (SD) | 28.83 (6.77) | 30.42 (7.97) | 0.214 |
| Current smoker, n (weighted %) | 1,451 (45.6) | 265 (34.6) | 0.227 |
| Alcohol (yes), n (weighted %) | 424 (12.2) | 97 (12.2) | 0.002 |
| Physically active, n (weighted %) | 4,120 (56.7) | 563 (41.9) | 0.300 |
| Diabetes, n (weighted %) | 902 (6.6) | 680 (27.5) | 0.580 |
| Hypertension, n (weighted %) | 2,951 (26.2) | 1,324 (58.6) | 0.694 |
| CVD history, n (weighted %) | 647 (5.0) | 532 (21.8) | 0.510 |

```
cat("\n*Table 1 note:* Weighted percentages use the NHANES complex survey design (strata, PSU, combined
    "Part A = analytic sample included in qgcomp complete-case modeling. Part B = analytic-sample partic
    "SMD in Part B is computed between excluded No CKD vs excluded CKD; values >0.10 are typically consi
```

```
##
## *Table 1 note:* Weighted percentages use the NHANES complex survey design (strata, PSU, combined weig
```

```
# Table 2: overall mixture effect
tab2 <- safe_qgcomp_extract(
  fit_qg_overall,
  "Model A: Overall mixture effect",
  nrow(analytic_qg)
```

```r
) %>%
  mutate(
    beta = fmt(beta),
    conf.low = fmt(conf.low),
    conf.high = fmt(conf.high),
    p.value = fmt_p(p.value),
    interpretation = "Change in PhenoAge acceleration (years)",
    note = if (is.null(fit_qg_overall)) {
      ifelse(is.null(qgcomp_overall_error), "qgcomp failed (unknown error)", qgcomp_overall_error)
    } else {
      "OK"
    }
  )
kable(
  tab2,
  caption = paste0(
    "Table 2. Descriptive overall association between environmental mixture score and ",
    "PhenoAge acceleration (years). ",
    "Mixture score ranges from 1 to 4; a 1-unit increase represents a 1-quartile increase averaged ",
    "across the fixed primary PFAS set. Estimates from qgcomp with normalized pooled NHANES sampling wei
  )
)
```

Table 28: Table 2. Descriptive overall association between environmental mixture score and PhenoAge acceleration (years). Mixture score ranges from 1 to 4; a 1-unit increase represents a 1-quartile increase averaged across the fixed primary PFAS set. Estimates from qgcomp with normalized pooled NHANES sampling weights.

| model | beta | conf.low | conf.high | p.value | N | interpretation | note |
|---|---|---|---|---|---|---|---|
| Model A: Overall mixture effect | -1.195 | -1.540 | -0.850 | <0.001 | 3058 | Change in PhenoAge acceleration (years) | OK |

```r
cat("\n*Table 2 note:* qgcomp model adjusted for same 6 variables as Model B: sex, race/ethnicity, PIR,
    "qgcomp intended covariates: ", paste(qg_covars_intended, collapse = ", "), ". ",
    "qgcomp realized covariates (after complete-case support checks): ", paste(qg_covars_realized, colla
    "This table is descriptive/directional; primary inference is based on Table 3 survey-weighted intera
    "PhenoAge acceleration defined as survey-weighted residual of PhenoAge on chronological age. ",
    "N reflects unweighted analytic sample used in mixture modeling.\n", sep = "")
```

```
##
## *Table 2 note:* qgcomp model adjusted for same 6 variables as Model B: sex, race/ethnicity, PIR, BMI
```

```r
# Table 3: interaction results + design-based marginal effects by CKD group
if (!is.null(fit_int)) {
  int_tidy <- broom::tidy(fit_int, conf.int = TRUE)
  cf_names <- names(coef(fit_int))

  int_terms_wanted <- c("mixture_qscore", cf_names[str_detect(cf_names, "^ckd$|mixture_qscore:ckd")])
  tab3_raw <- int_tidy %>%
    filter(term %in% int_terms_wanted) %>%
```

```r
  transmute(
    term,
    beta = fmt(estimate),
    conf.low = fmt(conf.low),
    conf.high = fmt(conf.high),
    p.value = fmt_p(p.value),
    N = n_modelb,
    model = "Model B: Mixture*CKD interaction"
  )

# Design-based marginal effects via svycontrast with named contrast vectors.

# Robustly identify the interaction term regardless of factor coding.
int_name <- cf_names[str_detect(cf_names, "mixture_qscore:ckd")]
if (length(int_name) != 1) {
  warning("Could not uniquely identify mixture*ckd interaction term among: ",
          paste(cf_names, collapse = ", "), ". Skipping marginal effects.")
  tab3 <- tab3_raw
} else {
  c_ckd0 <- setNames(rep(0, length(cf_names)), cf_names)
  c_ckd0["mixture_qscore"] <- 1

  c_ckd1 <- setNames(rep(0, length(cf_names)), cf_names)
  c_ckd1["mixture_qscore"] <- 1
  c_ckd1[int_name] <- 1

  contrast_ckd0 <- svycontrast(fit_int, c_ckd0)
  contrast_ckd1 <- svycontrast(fit_int, c_ckd1)

  ddf <- fit_int$df.residual
  if (is.null(ddf) || !is.finite(ddf)) ddf <- Inf

  make_contrast_row <- function(ctr, label, df) {
    est <- as.numeric(coef(ctr))
    se  <- as.numeric(SE(ctr))
    t_val <- est / se
    p_val <- 2 * pt(-abs(t_val), df = df)
    tibble(term = label, beta = fmt(est),
           conf.low = fmt(est - qt(0.975, df) * se),
           conf.high = fmt(est + qt(0.975, df) * se),
           p.value = fmt_p(p_val), N = n_modelb, model = "Marginal effects (design-based)")
  }

  p_int_tidy <- int_tidy %>% filter(term == int_name) %>% pull(p.value)
  p_interaction <- if (length(p_int_tidy) > 0) p_int_tidy[1] else NA_real_

  tab3_marginal <- bind_rows(
    make_contrast_row(contrast_ckd0, "Mixture effect among CKD=0", ddf),
    make_contrast_row(contrast_ckd1, "Mixture effect among CKD=1", ddf),
    tibble(term = "P-interaction", beta = "", conf.low = "", conf.high = "",
           p.value = fmt_p(p_interaction), N = n_modelb, model = "Marginal effects (design-based)")
  )
  tab3 <- bind_rows(tab3_raw, tab3_marginal)
```

```r
  }
} else {
  tab3 <- tibble(
    term = c("mixture_qscore", "ckd", "mixture_qscore:ckd"),
    beta = "NA", conf.low = "NA", conf.high = "NA", p.value = "NA",
    N = n_modelb, model = "Model B: Mixture*CKD interaction (not estimable)"
  )
}
kable(tab3, caption = "Table 3. Interaction model with design-based marginal effects by CKD group")
```

Table 29: Table 3. Interaction model with design-based marginal effects by CKD group

| term | beta | conf.low | conf.high | p.value | N | model |
|---|---|---|---|---|---|---|
| mixture_qscore | -0.773 | -1.077 | -0.469 | <0.001 | 3058 | Model B: Mixture*CKD interaction |
| ckd | 9.716 | 6.255 | 13.177 | <0.001 | 3058 | Model B: Mixture*CKD interaction |
| mixture_qscore:ckd | -1.451 | -2.615 | -0.288 | 0.015 | 3058 | Model B: Mixture*CKD interaction |
| Mixture effect among CKD=0 | -0.773 | -1.077 | -0.469 | <0.001 | 3058 | Marginal effects (design-based) |
| Mixture effect among CKD=1 | -2.225 | -3.342 | -1.107 | <0.001 | 3058 | Marginal effects (design-based) |
| P-interaction | | | | 0.015 | 3058 | Marginal effects (design-based) |

```r
cat("\n*Table 3 note:* Model B is a survey-weighted linear model (`svyglm`) with interaction term `mixtu
    "Intended covariates: ", paste(intended_covars_int, collapse = ", "), ". ",
    "Realized covariates (after complete-case support checks): ", paste(realized_covars_int, collapse =
    "Marginal effects for CKD=0 and CKD=1 are estimated using design-based contrasts (`svycontrast`) wi
```

```
##
## *Table 3 note:* Model B is a survey-weighted linear model (‘svyglm‘) with interaction term ‘mixture_
```

```r
# Table 4: sensitivity analyses -- using same covariate set as primary models.
sens_covars_rhs <- paste(realized_covars_int, collapse = " + ")
make_rhs_formula <- function(outcome, main_terms, covar_rhs) {
  rhs <- paste(main_terms, collapse = " + ")
  if (!is.null(covar_rhs) && nzchar(covar_rhs)) rhs <- paste(rhs, covar_rhs, sep = " + ")
  as.formula(paste(outcome, "~", rhs))
}
fit_svy_with_covar_guard <- function(data_df, outcome, main_terms, covars_terms, label) {
  build_formula <- function(covs) {
    covar_rhs <- if (length(covs) > 0) paste(covs, collapse = " + ") else ""
    make_rhs_formula(outcome, main_terms, covar_rhs)
  }
  covs <- covars_terms
  intended <- covs
  f <- build_formula(covs)
  mf <- NULL
```

```
  repeat {
    mf <- tryCatch(model.frame(f, data = data_df, na.action = na.omit), error = function(e) NULL)
    if (is.null(mf) || nrow(mf) == 0) break
    factor_terms <- names(mf)[vapply(mf, is.factor, logical(1))]
    bad_factor_terms <- factor_terms[vapply(mf[factor_terms], function(x) nlevels(droplevels(x)) < 2, le
    removable <- intersect(covs, bad_factor_terms)
    if (length(removable) == 0) break
    message("Sensitivity [", label, "]: dropping collapsed factor term(s): ", paste(removable, collapse
    covs <- setdiff(covs, removable)
    f <- build_formula(covs)
  }
  if (is.null(mf) || nrow(mf) == 0) {
    return(list(
      fit = NULL, n_model = 0L, intended = intended, realized = covs,
      dropped = paste(setdiff(intended, covs), collapse = ", "),
      error = "No complete-case rows for model frame."
    ))
  }
  data_svy <- as.data.frame(data_df)
  for (v in c("phenoage_accel", "phenoage_nokid", "phenoage_nokid_both", "phenoage_accel_unweighted", "r
    if (v %in% names(data_svy)) data_svy[[v]] <- suppressWarnings(as.numeric(as.character(data_svy[[v]]
  }
  if ("ckd" %in% names(data_svy)) data_svy$ckd <- as.integer(data_svy$ckd)
  for (v in c("sex", "race_eth", "cycle")) {
    if (v %in% names(data_svy)) data_svy[[v]] <- factor(as.character(data_svy[[v]]))
  }
  data_svy$sex      <- droplevels(factor(data_svy$sex))
  data_svy$race_eth <- droplevels(factor(data_svy$race_eth))
  data_svy$cycle    <- droplevels(factor(data_svy$cycle))
  des <- tryCatch(
    svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined, nest = TRUE, data = data_svy),
    error = function(e) svydesign(id = ~1, strata = NULL, weights = ~weight_combined, data = data_svy)
  )
  err <- NULL
  fit <- tryCatch(
    svyglm(f, design = des, family = gaussian()),
    error = function(e) {
      err <<- conditionMessage(e)
      NULL
    }
  )
  list(
    fit = fit,
    n_model = nrow(mf),
    intended = intended,
    realized = covs,
    dropped = paste(setdiff(intended, covs), collapse = ", "),
    error = err
  )
}

sens_ckd_egfr <- qdat %>%
  mutate(ckd_sens = ifelse(!is.na(egfr), as.integer(egfr < 60), NA_integer_)) %>%
```

```
  filter(!is.na(ckd_sens))
fit_sens1_obj <- fit_svy_with_covar_guard(sens_ckd_egfr, "phenoage_accel", c("mixture_qscore", "ckd_sens
fit_sens1 <- fit_sens1_obj$fit

sens_pheno <- qdat
ph2_comp <- compute_phenoage_components(sens_pheno, include_scr = FALSE, include_alb = TRUE)
ph2 <- ph2_comp$phenoage
sens_pheno$phenoage_nokid <- NA_real_
keep2 <- which(!is.na(ph2) & !is.na(sens_pheno$age))
if (length(keep2) > 0) {
  fit_nokid <- lm(ph2[keep2] ~ sens_pheno$age[keep2])
  sens_pheno$phenoage_nokid[keep2] <- residuals(fit_nokid)
}
fit_sens2_obj <- fit_svy_with_covar_guard(sens_pheno, "phenoage_nokid", c("mixture_qscore", "ckd"), real
fit_sens2 <- fit_sens2_obj$fit

ph3_comp <- compute_phenoage_components(sens_pheno, include_scr = FALSE, include_alb = FALSE)
ph3 <- ph3_comp$phenoage
sens_pheno$phenoage_nokid_both <- NA_real_
keep3 <- which(!is.na(ph3) & !is.na(sens_pheno$age))
if (length(keep3) > 0) {
  fit_nokid_both <- lm(ph3[keep3] ~ sens_pheno$age[keep3])
  sens_pheno$phenoage_nokid_both[keep3] <- residuals(fit_nokid_both)
}
fit_sens3_obj <- fit_svy_with_covar_guard(sens_pheno, "phenoage_nokid_both", c("mixture_qscore", "ckd")
fit_sens3 <- fit_sens3_obj$fit

get_mix_row <- function(fit, label, n_model = NA_integer_) {
  if (is.null(fit)) return(tibble(sensitivity = label, estimate = NA_real_, conf.low = NA_real_, conf.hi
  tb <- broom::tidy(fit, conf.int = TRUE) %>% filter(term == "mixture_qscore")
  if (nrow(tb) == 0) return(tibble(sensitivity = label, estimate = NA_real_, conf.low = NA_real_, conf.h
  tb %>% mutate(sensitivity = label, N = n_model) %>% select(sensitivity, estimate, conf.low, conf.high
}

f_sens_uw <- make_rhs_formula("phenoage_accel_unweighted", c("mixture_qscore", "ckd"), sens_covars_rhs)
f_sens_both <- make_rhs_formula("phenoage_accel", c("mixture_qscore", "ckd"), sens_covars_rhs)
fit_sens_uw_obj <- fit_svy_with_covar_guard(qdat, "phenoage_accel_unweighted", c("mixture_qscore", "ckd
fit_sens_both_markers_obj <- fit_svy_with_covar_guard(qdat %>% filter(!is.na(egfr), !is.na(acr)), "phen
fit_sens_no_acute_obj <- fit_svy_with_covar_guard(qdat %>% filter(is.na(crp_acute) | crp_acute == 0L),
fit_sens_strict_obj <- fit_svy_with_covar_guard(qdat_strict, "phenoage_accel", c("mixture_qscore", "ckd

# Sensitivity: additional adjustment for cardiometabolic conditions potentially on the causal pathway.
covars_cardiometabolic <- c(
  realized_covars_int,
  if (valid_num("diabetes", qdat)) "diabetes",
  if (valid_num("hypertension", qdat)) "hypertension",
  if (valid_num("cvd", qdat)) "cvd"
)
fit_sens_cardiometabolic_obj <- fit_svy_with_covar_guard(qdat, "phenoage_accel", c("mixture_qscore", "c
fit_sens_cardiometabolic <- fit_sens_cardiometabolic_obj$fit

tab4 <- bind_rows(
  get_mix_row(fit_sens1, "CKD redefined as eGFR<60", fit_sens1_obj$n_model),
```

```r
  get_mix_row(fit_sens2, "PhenoAge excluding creatinine only", fit_sens2_obj$n_model),
  get_mix_row(fit_sens3, "PhenoAge excluding creatinine and albumin", fit_sens3_obj$n_model),
  get_mix_row(
    fit_sens_uw_obj$fit,
    "Unweighted residualized PhenoAgeAccel (sensitivity)",
    fit_sens_uw_obj$n_model
  ),
  get_mix_row(
    fit_sens_both_markers_obj$fit,
    "Restricted to participants with both eGFR and ACR",
    fit_sens_both_markers_obj$n_model
  ),
  get_mix_row(
    fit_sens_no_acute_obj$fit,
    "Exclude participants with CRP > 1 mg/dL",
    fit_sens_no_acute_obj$n_model
  ),
  get_mix_row(
    fit_sens_cardiometabolic,
    "Additional adjustment for diabetes, hypertension, CVD (potentially on causal pathway)",
    fit_sens_cardiometabolic_obj$n_model
  ),
  get_mix_row(
    fit_sens_strict_obj$fit,
    "Strict complete-case (qgcomp-compatible PFAS subset), same 6-variable adjustment",
    fit_sens_strict_obj$n_model
  )
) %>%
  transmute(
    sensitivity,
    beta = fmt(estimate),
    conf.low = fmt(conf.low),
    conf.high = fmt(conf.high),
    p.value = fmt_p(p.value),
    N,
    model = "Sensitivity"
  )
kable(tab4, caption = "Table 4. Sensitivity analyses (primary adjustment: sex, race, PIR, BMI, smoking,
```

Table 30: Table 4. Sensitivity analyses (primary adjustment: sex, race, PIR, BMI, smoking, cycle)

| sensitivity | beta | conf.low | conf.high | p.value | N | model |
|---|---|---|---|---|---|---|
| CKD redefined as eGFR<60 | -1.094 | -1.385 | -0.804 | <0.001 | 3058 | Sensitivity |
| PhenoAge excluding creatinine only | -0.869 | -1.144 | -0.593 | <0.001 | 3058 | Sensitivity |
| PhenoAge excluding creatinine and albumin | -0.698 | -0.960 | -0.437 | <0.001 | 3058 | Sensitivity |
| Unweighted residualized PhenoAgeAccel (sensitivity) | -1.048 | -1.353 | -0.743 | <0.001 | 3058 | Sensitivity |

| sensitivity | beta | conf.low | conf.high | p.value | N | model |
|---|---|---|---|---|---|---|
| Restricted to participants with both eGFR and ACR | -0.947 | -1.253 | -0.640 | <0.001 | 3013 | Sensitivity |
| Exclude participants with CRP > 1 mg/dL | -1.049 | -1.354 | -0.744 | <0.001 | 2952 | Sensitivity |
| Additional adjustment for diabetes, hypertension, CVD (potentially on causal pathway) | -1.030 | -1.344 | -0.717 | <0.001 | 2960 | Sensitivity |
| Strict complete-case (qgcomp-compatible PFAS subset), same 6-variable adjustment | -1.021 | -1.312 | -0.729 | <0.001 | 3058 | Sensitivity |

```
cat("\n*Table 4 note:* Primary and most sensitivity models use the lean adjustment set (sex, race/ethnic
    "One row adds diabetes, hypertension, and CVD with explicit labeling that these may be on the causal
    "Each row reports the `mixture_qscore` coefficient from a survey-weighted model (`svyglm`) with NHA
```

```
##
## *Table 4 note:* Primary and most sensitivity models use the lean adjustment set (sex, race/ethnicity
```

```
sens_diagnostics <- bind_rows(
  tibble(sensitivity = "CKD redefined as eGFR<60", N_model = fit_sens1_obj$n_model, dropped_covariates =
  tibble(sensitivity = "PhenoAge excluding creatinine only", N_model = fit_sens2_obj$n_model, dropped_co
  tibble(sensitivity = "PhenoAge excluding creatinine and albumin", N_model = fit_sens3_obj$n_model, dro
  tibble(sensitivity = "Unweighted residualized PhenoAgeAccel (sensitivity)", N_model = fit_sens_uw_obj$
  tibble(sensitivity = "Restricted to participants with both eGFR and ACR", N_model = fit_sens_both_mark
  tibble(sensitivity = "Exclude participants with CRP > 1 mg/dL", N_model = fit_sens_no_acute_obj$n_mode
  tibble(sensitivity = "Additional adjustment for diabetes, hypertension, CVD (potentially on causal pa
  tibble(sensitivity = "Strict complete-case (qgcomp-compatible PFAS subset), same 6-variable adjustmen
)
kable(sens_diagnostics, caption = "Sensitivity model diagnostics (dropped covariates and fit errors).")
```

Table 31: Sensitivity model diagnostics (dropped covariates and fit errors).

| sensitivity | N_model | dropped_covariates | fit_error |
|---|---|---|---|
| CKD redefined as eGFR<60 | 3058 | None | None |
| PhenoAge excluding creatinine only | 3058 | None | None |
| PhenoAge excluding creatinine and albumin | 3058 | None | None |
| Unweighted residualized PhenoAgeAccel (sensitivity) | 3058 | None | None |
| Restricted to participants with both eGFR and ACR | 3013 | None | None |
| Exclude participants with CRP > 1 mg/dL | 2952 | None | None |
| Additional adjustment for diabetes, hypertension, CVD (potentially on causal pathway) | 2960 | None | None |
| Strict complete-case (qgcomp-compatible PFAS subset), same 6-variable adjustment | 3058 | None | None |

```
# Supplementary: single-pollutant models with same 6-variable adjustment as Model B.
single_pollutant_out <- lapply(exposure_vars, function(v) {
  sp_obj <- fit_svy_with_covar_guard(qdat, "phenoage_accel", c(v, "ckd"), realized_covars_int, paste0("S
  fit_sp <- sp_obj$fit
  if (is.null(fit_sp)) {
    return(list(
```

```r
      row = tibble(exposure = v, beta = NA_real_, conf.low = NA_real_, conf.high = NA_real_, p.value = N
      diag = tibble(exposure = v, N_model = sp_obj$n_model, dropped_covariates = ifelse(sp_obj$dropped =
    ))
  }
  tb <- broom::tidy(fit_sp, conf.int = TRUE) %>% filter(term == v)
  if (nrow(tb) == 0) {
    return(list(
      row = tibble(exposure = v, beta = NA_real_, conf.low = NA_real_, conf.high = NA_real_, p.value = N
      diag = tibble(exposure = v, N_model = sp_obj$n_model, dropped_covariates = ifelse(sp_obj$dropped =
    ))
  }
  list(
    row = tibble(
      exposure = v,
      beta = tb$estimate[1],
      conf.low = if ("conf.low" %in% names(tb)) tb$conf.low[1] else NA_real_,
      conf.high = if ("conf.high" %in% names(tb)) tb$conf.high[1] else NA_real_,
      p.value = if ("p.value" %in% names(tb)) tb$p.value[1] else NA_real_,
      N = sp_obj$n_model
    ),
    diag = tibble(exposure = v, N_model = sp_obj$n_model, dropped_covariates = ifelse(sp_obj$dropped ==
  )
})
single_pollutant <- bind_rows(lapply(single_pollutant_out, `[[`, "row")) %>%
  mutate(
    beta = fmt(beta),
    conf.low = fmt(conf.low),
    conf.high = fmt(conf.high),
    p.value = fmt_p(p.value)
  )
single_pollutant_diagnostics <- bind_rows(lapply(single_pollutant_out, `[[`, "diag"))

kable(
  single_pollutant,
  caption = "Supplementary Table. Single-pollutant associations with PhenoAge acceleration (survey-weigh
)
```

Table 32: Supplementary Table. Single-pollutant associations with PhenoAge acceleration (survey-weighted, same 6-variable adjustment as Model B).

| exposure | beta | conf.low | conf.high | p.value | N |
|---|---|---|---|---|---|
| LBXPFOS_log2 | -0.925 | -1.312 | -0.538 | <0.001 | 3058 |
| LBXPFOA_log2 | -1.399 | -1.760 | -1.039 | <0.001 | 3058 |
| LBXPFNA_log2 | -0.626 | -0.947 | -0.305 | <0.001 | 3058 |
| LBXPFHS_log2 | -0.809 | -1.049 | -0.568 | <0.001 | 3058 |

```r
kable(single_pollutant_diagnostics, caption = "Supplementary model diagnostics (single-pollutant droppe
```

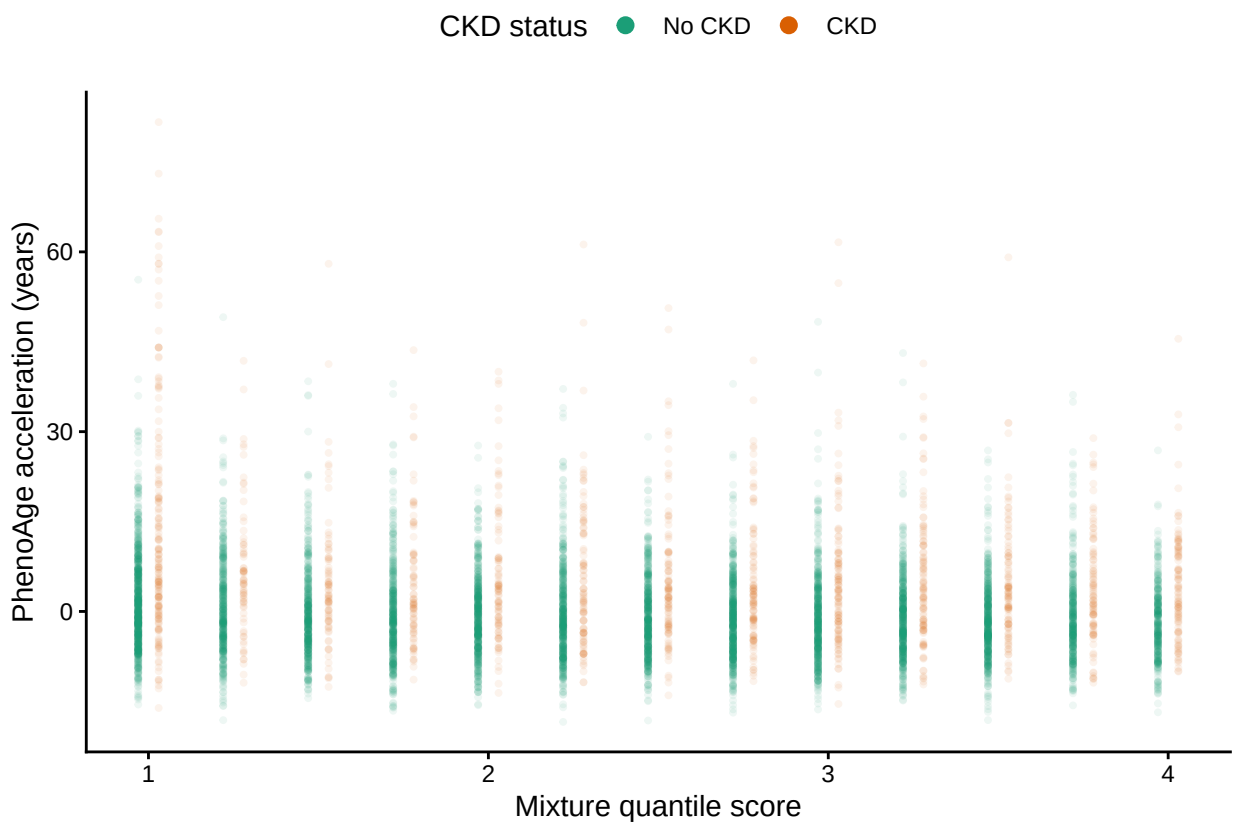
Table 33: Supplementary model diagnostics (single-pollutant dropped covariates and fit errors).

| exposure | N_model | dropped_covariates | fit_error |
|---|---|---|---|
| LBXPFOS_log2 | 3058 | None | None |
| LBXPFOA_log2 | 3058 | None | None |
| LBXPFNA_log2 | 3058 | None | None |
| LBXPFHS_log2 | 3058 | None | None |

# Figures

```
dir.create("figures", showWarnings = FALSE)
n_chems <- length(primary_exposure_vars)

# Figure 1: adjusted survey-weighted predicted lines by CKD status.
pred_lines <- tryCatch({
  pdat_list <- lapply(c(0L, 1L), function(ck) {
    sub_df <- qdat %>% filter(ckd == ck)
    svy_sub <- svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined, nest = TRUE, data = su
    fit_pred <- svyglm(phenoage_accel ~ mixture_qscore, design = svy_sub, family = gaussian())
    score_seq <- seq(min(sub_df$mixture_qscore, na.rm = TRUE),
                     max(sub_df$mixture_qscore, na.rm = TRUE), length.out = 100)
    newdat <- data.frame(mixture_qscore = score_seq)
    preds <- predict(fit_pred, newdata = newdat, se.fit = TRUE, type = "response")
    newdat$fit <- as.numeric(preds$fit)
    newdat$se <- as.numeric(preds$se.fit)
    newdat$ckd <- ifelse(ck == 1L, "CKD", "No CKD")
    newdat
  })
  bind_rows(pdat_list)
}, error = function(e) NULL)

# Overall marginal direction: as mixture increases, what happens to phenoage_accel (pooled model).
overall_direction <- tryCatch({
  svy_full <- svydesign(id = ~psu, strata = ~strata, weights = ~weight_combined, nest = TRUE, data = qda
  fit_overall <- svyglm(phenoage_accel ~ mixture_qscore, design = svy_full, family = gaussian())
  score_seq <- seq(min(qdat$mixture_qscore, na.rm = TRUE), max(qdat$mixture_qscore, na.rm = TRUE), leng
  newdat <- data.frame(mixture_qscore = score_seq)
  preds <- predict(fit_overall, newdata = newdat, se.fit = TRUE, type = "response")
  newdat$fit <- as.numeric(preds$fit)
  newdat
}, error = function(e) NULL)

# Horizontal offset so green (No CKD) and orange (CKD) dots don't overlap at each quantile
x_range <- diff(range(qdat$mixture_qscore, na.rm = TRUE))
pt_offset <- max(0.005, x_range * 0.01)
qdat_plot <- qdat %>%
  mutate(
    x_plot = mixture_qscore + ifelse(ckd == 0L, -pt_offset, pt_offset),
    ckd_label = factor(ifelse(ckd == 0L, "No CKD", "CKD"), levels = c("No CKD", "CKD"))
```

```r
  )

fig1 <- ggplot(qdat_plot, aes(x = x_plot, y = phenoage_accel)) +
  geom_point(aes(color = ckd_label), alpha = 0.08, size = 0.7) +
  labs(
    x = "Mixture quantile score",
    y = "PhenoAge acceleration (years)",
    color = "CKD status"
  ) +
  scale_color_manual(values = c("No CKD" = "#1b9e77", "CKD" = "#d95f02")) +
  guides(color = guide_legend(override.aes = list(alpha = 1, size = 2.5))) +
  theme_classic(base_size = 11) +
  theme(legend.position = "top")

if (!is.null(pred_lines)) {
  fig1 <- fig1 +
    geom_ribbon(
      data = pred_lines,
      aes(x = mixture_qscore, ymin = fit - 1.96 * se, ymax = fit + 1.96 * se, fill = ckd),
      alpha = 0.2, inherit.aes = FALSE
    ) +
    geom_line(
      data = pred_lines,
      aes(x = mixture_qscore, y = fit, color = ckd),
      linewidth = 1.0, inherit.aes = FALSE
    ) +
    scale_fill_manual(values = c("No CKD" = "#1b9e77", "CKD" = "#d95f02")) +
    scale_color_manual(values = c("No CKD" = "#1b9e77", "CKD" = "#d95f02"))
}
# Dotted line: overall direction of association (as mixture increases, phenoage_accel changes)
if (!is.null(overall_direction)) {
  fig1 <- fig1 +
    geom_line(
      data = overall_direction,
      aes(x = mixture_qscore, y = fit),
      linetype = "dotted", color = "gray25", linewidth = 0.9, inherit.aes = FALSE
    )
}
fig1
```




```
ggsave("figures/fig1_mixture_phenoage.jpg", plot = fig1, width = 8, height = 6, dpi = 300, device = "jp

# Figure 2: forest plot of marginal mixture effects (design-based).
fig2 <- NULL
if (!is.null(fit_int)) {
  cf_names <- names(coef(fit_int))
  int_name <- cf_names[str_detect(cf_names, "mixture_qscore:ckd")]
  if (length(int_name) == 1 && "mixture_qscore" %in% cf_names) {
    c_ckd0 <- setNames(rep(0, length(cf_names)), cf_names); c_ckd0["mixture_qscore"] <- 1
    c_ckd1 <- setNames(rep(0, length(cf_names)), cf_names); c_ckd1["mixture_qscore"] <- 1; c_ckd1[int_na
    ctr0 <- svycontrast(fit_int, c_ckd0)
    ctr1 <- svycontrast(fit_int, c_ckd1)
    ddf <- fit_int$df.residual; if (is.null(ddf) || !is.finite(ddf)) ddf <- Inf

    mk_row <- function(ctr, grp) {
      est <- as.numeric(coef(ctr)); se <- as.numeric(SE(ctr))
      tibble(
        group = grp,
        estimate = est,
        conf.low = est - qt(0.975, ddf) * se,
        conf.high = est + qt(0.975, ddf) * se,
        p.value = 2 * pt(-abs(est / se), df = ddf)
      )
    }
    forest_df <- bind_rows(
      mk_row(ctr0, "No CKD"),
```

```
      mk_row(ctr1, "CKD")
    ) %>%
      mutate(group = factor(group, levels = c("No CKD", "CKD")))

    fig2 <- ggplot(forest_df, aes(x = estimate, y = group, color = group)) +
      geom_vline(xintercept = 0, linetype = "dashed", color = "gray40") +
      geom_errorbarh(aes(xmin = conf.low, xmax = conf.high), height = 0.18, linewidth = 0.8) +
      geom_point(size = 2.8) +
      geom_text(aes(label = paste0("p=", format(round(p.value, 3), nsmall = 3))),
                nudge_y = 0.2, size = 3, color = "black", show.legend = FALSE) +
      scale_color_manual(values = c("No CKD" = "#1b9e77", "CKD" = "#d95f02")) +
      labs(
        title = "Marginal mixture effects by CKD status",
        subtitle = "Design-based contrasts from interaction model",
        x = "Beta (95% CI) for mixture score",
        y = NULL
      ) +
      theme_classic(base_size = 11) +
      theme(legend.position = "none")
    fig2
    ggsave("figures/fig2_forest_marginal_effects.jpg", plot = fig2, width = 7, height = 4, dpi = 300, d
  }
}
```

## Cautions and Limitations

- Cross-sectional NHANES data do not establish temporality.
- PhenoAge includes kidney-related biomarkers (creatinine, albumin), creating potential circularity with CKD definitions. Sensitivity analyses recompute PhenoAge excluding (a) creatinine only and (b) both creatinine and albumin; both are non-standard derivatives and interpreted as robustness checks.
- PhenoAge uses Levine (2018) coefficients with correct units: albumin g/L (NHANES g/dL × 10), creatinine µmol/L (mg/dL × 88.4), glucose mmol/L (mg/dL ÷ 18.018). Model B uses phenoage_accel (age-residualized) and does not adjust for age again. Mixture quantiles are computed within each NHANES cycle to avoid temporal confounding (PFAS trends 2003–2018). Survey design uses full PSU/strata when possible; if the design fails (e.g. singleton PSUs), a weights-only fallback is used and noted.
- `qgcomp` point estimates are presented descriptively (pooled sampling weights applied, but PSU/strata not incorporated for variance estimation). Statistical inference relies on the survey-weighted regression models (`svyglm`) above. Bootstrap variance estimation incorporating replicate weights was not implemented because NHANES does not provide replicate weights for environmental subsamples across all cycles.
- LOD handling follows NHANES detection-flag conventions when indicator columns are available (`*LC` or `*L`): values flagged below LOD (indicator == 1) are replaced with value / sqrt(2). When no LOD indicator is available, values are left unchanged (no fallback NA imputation), which improves transparency but can reduce complete-case sample size.
- **Weight selection**: a weight audit table is printed and `config$weight_type = "auto"` applies the top-coverage eligible weight. For PFAS, auto-selection excludes full-sample interview/MEC weights (`WTINT2YR`, `WTMEC2YR`) and prioritizes environmental subsample weights (`WTS*2YR`), including the combined cross-cycle PFAS weight column.
- The primary PFAS mixture was defined as a fixed four-chemical set available across all retained cycles (PFOS, PFOA, PFNA, and PFHxS), with complete-case requirement on that set. PFDA was not consistently available across cycles and was therefore excluded from the primary mixture and evaluated

only in supplementary analyses. Strict complete-case (qgcomp-compatible PFAS subset) modeling is reported as sensitivity. A sensitivity analysis using an available-exposure (adaptive) mixture can be added to assess robustness when using broader PFAS information.

- Exposure quantile cutpoints are defined within cycle using cycle-specific NHANES sampling weights.
- PhenoAgeAccel is computed as the residual from a survey-weighted regression of PhenoAge on chronological age (`svyglm`), ensuring the construct is population-representative. An unweighted residualization is included as sensitivity.
- CRP unit detection uses a two-part rule: (1) table-name provenance (HSCRP tables imply mg/L), and (2) a conservative distribution heuristic (median > 1.5 **and** p75 > 3). If either triggers, CRP is converted from mg/L to mg/dL (/ 10). Cycle-level diagnostic tables are printed. PhenoAge is recomputed after any conversions. Glucose is converted from mmol/L to mg/dL if median < 6.
- Extreme biomarker values are handled by plausibility filters (WBC > 50, glucose > 600, ALP > 500 set to missing). CRP > 1 mg/dL is retained in primary analyses and evaluated in sensitivity analyses excluding potential acute inflammatory states.
- Glucose is sourced from MEC biochemistry (`glu_mec`: BIOPRO/L40) for the primary analysis. The separate GLU table (fasting) is joined with renamed variables (`LBXGLU_GLU`, `LBXSGL_GLU`) to avoid duplicate columns; it is retained for diagnostics but not used in the primary model.
- Marginal effects in the interaction model (Table 3) use `svycontrast()` with design-based degrees of freedom and t-distribution p-values, maintaining full complex-survey consistency.
- Because complete-case requirements can collapse factor levels in the interaction model frame, Model B reports both intended and realized covariate sets; unsupported factor terms are dropped with explicit logging.

# Reproducibility

```r
sessionInfo()
```

```
## R version 4.5.3 (2026-03-11 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] splines   grid      stats     graphics  grDevices utils     datasets
## [8] methods   base
##
## other attached packages:
##  [1] tibble_3.3.1   forcats_1.0.1  qgcomp_2.18.10 knitr_1.51     broom_1.0.12
##  [6] ggplot2_4.0.2  survey_4.5     survival_3.8-6 Matrix_1.7-4   stringr_1.6.0
```

```
## [11] purrr_1.2.1    tidyr_1.3.2    dplyr_1.2.1    nhanesA_1.4.1
##
## loaded via a namespace (and not attached):
##  [1] gtable_0.3.6        xfun_0.57           lattice_0.22-9
##  [4] numDeriv_2016.8-1.1 vctrs_0.7.2         tools_4.5.3
##  [7] Rdpack_2.6.6        generics_0.1.4      curl_7.0.0
## [10] sandwich_3.1-1      pkgconfig_2.0.3     arm_1.15-2
## [13] RColorBrewer_1.1-3  S7_0.2.1            rootSolve_1.8.2.4
## [16] lifecycle_1.0.5     compiler_4.5.3      farver_2.1.2
## [19] textshaping_1.0.5   mitools_2.4         carData_3.0-6
## [22] htmltools_0.5.9     yaml_2.3.12         Formula_1.2-5
## [25] car_3.1-5           pillar_1.11.1       nloptr_2.2.1
## [28] MASS_7.3-65         reformulas_0.4.4    boot_1.3-32
## [31] abind_1.4-8         nlme_3.1-168        tidyselect_1.2.1
## [34] rvest_1.0.5         digest_0.6.39       stringi_1.8.7
## [37] AER_1.2-16          labeling_0.4.3      fastmap_1.2.0
## [40] cli_3.6.6           magrittr_2.0.5      withr_3.0.2
## [43] foreign_0.8-91      scales_1.4.0        backports_1.5.1
## [46] rmarkdown_2.31      httr_1.4.8          otel_0.2.0
## [49] nnet_7.3-20         lme4_2.0-1          gridExtra_2.3
## [52] ragg_1.5.2          zoo_1.8-15          coda_0.19-4.1
## [55] evaluate_1.0.5      rbibutils_2.4.1     lmtest_0.9-40
## [58] pscl_1.5.9          rlang_1.2.0         Rcpp_1.1.1
## [61] glue_1.8.0          DBI_1.3.0           selectr_0.5-1
## [64] xml2_1.5.2          minqa_1.2.8         R6_2.6.1
## [67] plyr_1.8.9          systemfonts_1.3.2
```